

An improved privacy preserving authentication protocol with fast reconnection for WLAN-IOT devices

Vijay Kumar^{*}, Aditya Kaushal Ranjan, Prabhat Kumar

Department of Computer Science and Engineering, National Institute of Technology Patna, India

ARTICLE INFO

Keywords:

Extensible Authentication Protocol (EAP)
PUF
Privacy
Authentication
Wireless local area networks (WLANs)

ABSTRACT

The rapid development of Internet of Things (IoT) devices in everyday life promotes their access to the Internet, hence increasing the vulnerability to security risks. Developing a secure authentication protocol to protect Internet of Things (IoT) devices as they integrate more into Wireless Local Area Networks (WLANs) presents a significant challenge in addressing new cyber threats while ensuring reliable connectivity. Many symmetric and asymmetric encryption protocols, intended to protect WLAN-connected IoT devices, are susceptible to various attacks. This vulnerability leads to significant communication and computational overheads, making them unsuitable for Internet of Things devices with limited resources. Such protocols are prone to threats like perfect forward secrecy, ephemeral secret leakage, insider attacks, stolen device attacks, and traceability attacks, along with the additional burden of security encryption costs. We present a simple authentication method based on the Extensible Authentication Protocol (EAP) framework and Elliptic Curve Cryptography (ECC) for mutual authentication with key agreement for the IoT in a WLAN environment. The primary feature of the proposed protocol lies in its hybrid integrated design that combines ECC-based secure registration over public channels, lightweight symmetric-key mutual authentication, and a PUF-enhanced fast reconnection mechanism, all within a unified framework that provides strong security guarantees, including perfect forward secrecy and resilience against insider attacks, while maintaining low overhead suitable for resource-constrained IoT devices. A comprehensive security analysis, both formal and informal, demonstrates the protocol's resilience against known and emerging threats. Validation using the RoR oracle model and Scyther verification tool confirms the robustness of the protocol under various attack scenarios. Performance evaluation reveals that the proposed solution achieves superior security with reduced computational and communication overheads compared to state-of-the-art protocols, making it highly suitable for IoT applications. This work provides a scalable, efficient, and secure framework for protecting IoT devices in modern WLAN deployments.

1. Introduction

The Internet of Things (IoT) has transitioned from an innovative idea to a fundamental cornerstone of an intelligent future. It facilitates a comprehensive ecosystem of interconnected devices equipped with sensors, software, and communication technologies to create seamless, intelligent networks. The principal objective of IoT is to enable automated, data-driven decision-making and improve user experiences via seamless interaction between humans and machines [1]. The Internet of Things (IoT) has transformed our interaction with environments by improving data collection and exchange through the internet and various communication networks, thereby cultivating a more intelligent and responsive world.

At present, the Internet of Things (IoT) is deeply integrated into routine life via applications including wearable health monitors, smartwatches, interconnected home appliances, industrial machinery, and autonomous vehicles. The effect of it reaches essential areas such as smart homes, smart grids, smart cities, and advanced healthcare systems. Moreover, IoT has changed sectors including manufacturing, hospitality, and tourism, underscoring its growing significance in economic and societal roles [2].

Nowadays, affordable Wireless Local Area Networks (WLANs) have emerged as an essential facilitator of IoT expansion, accommodating dense deployments of interconnected devices. However, because wireless communication is transmitted over public broadcasting frequencies, it is inherently open, giving enemies access to an attack surface.

^{*} Corresponding author.

E-mail addresses: vijayk.ph21.cs@nitp.ac.in (V. Kumar), aditya.cs18@nitp.ac.in (A.K. Ranjan), prabhat@nitp.ac.in (P. Kumar).

<https://doi.org/10.1016/j.jisa.2026.104445>

Malicious entities can eavesdrop on, intercept, or modify communications between users and servers, rendering WLAN security a critical concern.

Authentication is paramount in this security imperative, providing as the essential mechanism for verifying devices and users to ensure that only authorised entities can access the network. In sensitive environments like smart home ecosystems and medical IoT systems, where personal health data and private information are frequently shared, efficient authentication supersedes basic technical protocols. It is essential for protecting individual privacy, sustaining user trust, and ensuring system integrity. Absent such protection, the connectivity that fuels innovation simultaneously becomes a conduit for significant vulnerability.

Designing a secure, privacy-preserving communication scheme for IoT devices in WLAN environments one that also supports fast, reliable reconnection—remains a formidable challenge [3]. While the IEEE 802.11i standard provides a security architecture for WLANs, including key management and authentication through the IEEE 802.1X framework with the Extensible Authentication Protocol (EAP) [4,5], many existing EAP-based and other cryptographic methods exhibit critical vulnerabilities. These include weaknesses against perfect forward secrecy violations, ephemeral secret leakage, insider threats, device theft, and traceability attacks, often while imposing excessive computational and communication costs [6]. Consequently, there is a clear and pressing need for an authentication protocol that not only integrates seamlessly into the EAP-WLAN architecture but also overcomes these pervasive security shortcomings without sacrificing efficiency especially for resource-constrained IoT deployments where both security and performance are non-negotiable.

1.1. Contribution and motivations

Securing IoT device authentication on Wireless Local Area Networks (WLAN) presents a significant challenge due to the inherent limitations of the devices and the insufficiency of current solutions. Symmetric and asymmetric cryptography have been used to create various protocols, mainly based on the Extensible Authentication Protocol (EAP) framework. Asymmetric approaches, although resilient, are computationally costly for resource-constrained Internet of Things (IoT) devices. On the other hand, symmetric-key protocols are efficient and rapid but current cryptanalysis indicates that they are significantly susceptible to vulnerabilities. They often lack essential security attributes, including identity protection, perfect forward secrecy (PFS), and resistance to risks such as ephemeral secret leakage (ESL), insider threats, and device capture. Furthermore, they frequently lack domain. Critical features such as fast reconnection for mobile devices and rely on an initial secure registration channel, an assumption that is impractical in several public IoT deployment contexts.

This scenario reveals a distinct deficiency: no existing protocol concurrently fulfils the severe cost and efficiency demands of IoT devices while providing robust, proven security for WLAN environments. Consequently, this work is motivated by the urgent need to develop a practical, resilient, and efficient authentication mechanism tailored for this environment.

This study primarily contributes to the design and analysis of a new lightweight authentication protocol for IoT devices within WLAN. While individual cryptographic components are well-established, the novelty and significance of our work lies in their specific integration, optimisation, and enhancement to create a holistic solution. Our protocol uniquely combines:

- 1) The proposed protocol uses symmetric key cryptography for authentication and key agreement.
- 2) The registration phase of the suggested method does not require a secure channel. The secure channel is challenging to deploy in practical situations. A highly effective approach is a technique based

on public key cryptography. During the registration phase, smart devices are capable of handling a one-time public key-based procedure. This strategy is crucial for privacy preservation in smart homes and smart hospitals, where gadgets operate autonomously without user intervention. Establishing a secure route in public infrastructures such as smart hospitals is extremely challenging. Expensive ECC processes are restricted to a singular registration by using a hybrid paradigm that facilitates secure registration without a previously established secure channel via PUF and dual-control passwords. It ensures that fast reconnection does not compromise forward secrecy by leveraging Physical Unclonable Functions (PUF) and ephemeral nonces. It embeds systematic DoS protection throughout the protocol lifecycle. This combined approach offers a practical and secure solution not achieved by prior schemes focusing on individual aspects.

- 3) The formal and informal security evaluations validate that the proposed protocol delivers the stated security characteristics and can produce the requisite security credentials.
- 4) The empirical performance analysis and comparison demonstrate that the suggested protocol improves existing protocols for computational cost, communication cost, and energy consumption.

In summary, we connect the essential gap between theoretical security and practical implementation, providing a protocol that is both provably secure and genuinely implementable on low-cost IoT hardware within real WLAN systems.

2. Related work

2.1. PKI based EAP protocols

Public-key-based EAP methods rely on asymmetric cryptography for session key establishment and message security, typically necessitating certificate-based trust. EAP-Transport Layer Security (EAP-TLS) [7] offers strong security but imposes considerable message overhead and computational expense, rendering it impractical for resource-limited IoT devices. EAP-Tunneled TLS (EAP-TTLS) [8] reduces client-side requirements by eliminating the need for client certificates, yet it still depends on server-side PKI and remains vulnerable to several threats in IoT contexts, such as traceability, stolen device attacks, and ephemeral key leakage. Even recent asymmetric proposals, such as that by Yadav et al. [9], have been shown by Guo et al. [10] that it fails to provide protection against session key security, perfect forward secrecy, resistance to impersonation attacks and offline password guessing attacks.

2.2. Symmetric key based EAP protocol

Symmetric-key EAP protocols utilize pre-shared secrets to achieve efficient authentication with lower computational demands [11]. Cheikhrouhou et al. [4] introduced a hash-based symmetric method that enables mutual authentication with minimal messaging, though it lacks support for fast reconnection and remains vulnerable to replay and forward secrecy attacks. Subsequent symmetric proposals, such as those by Amit et al. [12] and Yadav et al. [13,14], improve efficiency but fail to address critical threats including traceability, insider attacks, and short-term key leakage. Beyond EAP frameworks, several lightweight IoT authentication schemes have been developed, such as the protocol by Rana et al. [15], yet these often sacrifice user anonymity and resilience against physical device capture, as demonstrated by Chen et al. [16] in their analysis of Alzahrani et al.'s scheme [17]. Guo et al. [18] further systematize these weaknesses, showing that many lightweight designs remain exposed to implicit attack vectors common in IoT deployments, underscoring the need for integrated solutions that balance efficiency with comprehensive security.

2.3. Research gaps

We found a number of enduring issues with current EAP-based and lightweight authentication protocols that prevent their widespread use in IoT-WLAN settings. These deficiencies collectively demonstrate the necessity for a more comprehensive and resource-conscious security framework.

1. **Dependence on Secure Initial Channels:** A significant number of symmetric-key schemes [4,8,12–17,19–22] fundamentally rely on the availability of a secure, out-of-band channel for the initial registration of devices and users. This assumption is often impractical in large-scale or public IoT deployments, creating a critical bootstrap vulnerability.
2. **Inadequate Identity Protection and Anonymity:** Many protocols [8,12–15,20,21,23–25] transmit device or user identities in plaintext or using static pseudonyms, failing to provide robust anonymity. This exposes communication patterns and device activities to passive eavesdroppers.
3. **Susceptibility to Traceability Attacks:** Linked to the lack of anonymity, several schemes [8,12–14,20,21,24,25] are vulnerable to traceability attacks, where an adversary can correlate different protocol sessions to track a specific device's movements or usage patterns over time.
4. **Lack of Resistance to Insider Threats:** A common oversight is the failure to defend against privileged insider attacks. Protocols such as [4,13,20], and [25] do not incorporate mechanisms to mitigate risks posed by malicious actors within the trusted network domain who may have access to long-term system parameters.
5. **Vulnerability to Ephemeral Secret Leakage (ESL):** Numerous designs [12,13,21] lack safeguards against the compromise of temporary session-state secrets. If these ephemeral values are exposed, they can be exploited to derive session keys, breaking session confidentiality.
6. **Absence of Perfect Forward Secrecy (PFS):** Several important protocols [8,12–14,20,21,23] do not guarantee perfect forward secrecy. Consequently, the compromise of a single long-term secret key could lead to the retrospective decryption of all past communications secured under that key.
7. **Missing Fast Reconnection Capability:** For IoT devices that frequently roam or experience intermittent connectivity, the lack of a fast reauthentication mechanism—a flaw present in many protocols [8,13,20,21,23,25] results in excessive latency and computational

overhead during reconnection, degrading user experience and network efficiency.

3. Preliminaries and backgrounds

3.1. Security requirement and design goal: building a trustworthy connection

The security goals that must be met for designing the protocol are as follows: -

When we design the protocol for your device and our server to talk, our primary job is to build a foundation of absolute trust. Here are the core promises we make to ensure every interaction is secure and private:

- 1) **We Will Introduce Ourselves Properly (Mutual Authentication)**
Before any secret is shared, your device and our server will formally verify each other's identity. It's like a digital handshake where both parties confirm, "Yes, I know who you are, and you can trust me."
- 2) **We Will Keep Your Identity Confidential (Identity & Anonymity)**
We promise your real identity won't be broadcast in the open. Think of it like using a secure, private meeting room instead of shouting across a public square. Even if someone is listening, they won't know who is talking.
- 3) **We Will Make Your Sessions Untraceable (Unlinkability)**
An outside observer shouldn't be able to connect your different conversations with us. Each new session will look unique and unrelated, protecting your patterns of activity from being pieced together.
- 4) **We Will Spot and Reject Old Messages (Replay Attack Resistance)**
If someone tries to resend a message they eavesdropped on earlier, our system will recognise it as a stale copy and reject it. Every conversation is fresh and unique.
- 5) **We Will Shut Out Eavesdroppers (Man-in-the-Middle Resistance)**
We design the conversation so that no third party can secretly sit in the middle, pretending to be you to the server and the server to you. The connection is direct and tamper-proof.
- 6) **We Will Limit Damage from Leaked Temporary Data (Ephemeral Secret Safety)**
Even if some temporary, single-use data is accidentally exposed, it won't be enough for an attacker to crack the code and access your session.
- 7) **We Will Protect Your Past Conversations (Perfect Forward Secrecy)**

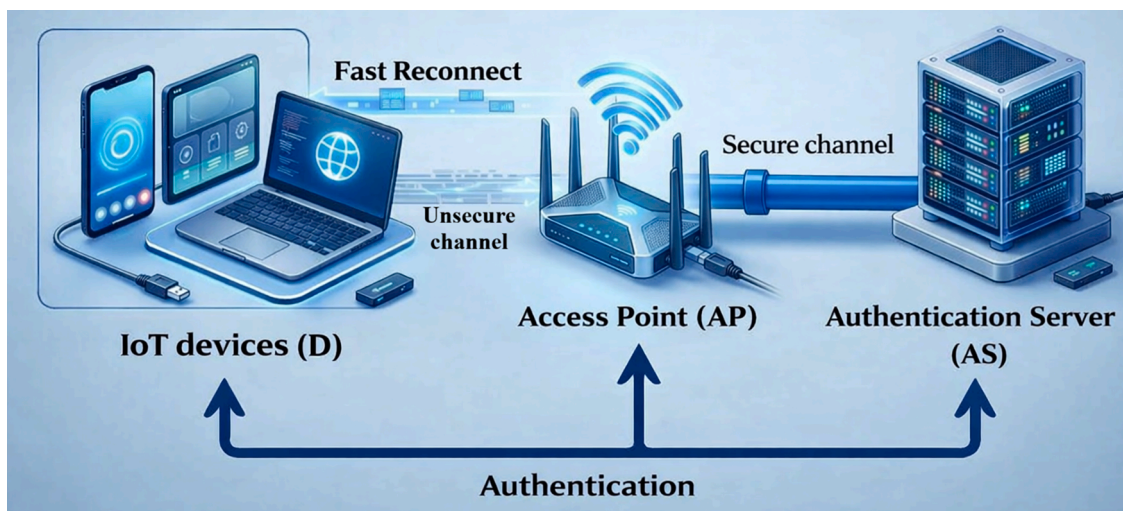


Fig. 1. Network model.

In the unlikely event our long-term master keys are compromised, all your past conversations remain locked and private. A breach today cannot decrypt yesterday's secrets.

8) We Will Guard Data on Lost Devices (Stolen Device Protection)

If your physical device is lost or stolen, the secrets stored on it are safeguarded and should not be easily extractable, preventing impersonation.

9) We Will Recover Gracefully from Desynchronisation (Desynchronisation Protection)

If an attacker tries to disrupt the authentication process midway, it may fail on that occasion, but the system will remain intact. We assume that authorised personnel within our organisation could pose a threat. Your next attempt to connect will work perfectly.

10) We Will Resist Overload Attempts (DoS Protection)

Our server is designed to identify and ignore massive floods of fake or repeated connection requests, ensuring real users can always get through.

11) We Will Verify Identities Beyond Doubt (Impersonation Resistance)

Simply having a stolen username or partial data won't be enough for an attacker to pretend to be you. The protocol requires proof that only the real you or the real server can provide.

12) We will defend you from within (Privileged Insider Resistance)

because we believe that authorised individuals within our organisation may pose a threat. Consequently, the system is structured to prevent employees or administrators from readily misusing their access to impersonate users or appropriate their credentials.

3.2. Network model

Wireless Local Area Network (WLAN) technology enables network services within a defined coverage area, offering a practical solution for connecting IoT devices and user equipment. Its ease of deployment, scalability, and support for mobility make it well-suited for environments where devices require seamless connectivity while moving within the network range. WLAN infrastructure allows devices to maintain active connections and supports the dynamic addition of new nodes without significant reconfiguration [26]. Fig. 1 illustrated the network model for the WLAN-IoT framework [27].

There are following consists of three core entities involved in communications as:

1) **IoT Devices (D)**

- a. These are endpoint devices such as smartphones, wearables, tablets, sensors, and embedded systems.
- b. They initiate connection requests and communicate over wireless channels.

2) **Access Point (AP)**

- a. The AP acts as a gateway, bridging IoT devices with the backend Authentication Server (AS).
- b. It provides initial connectivity, manages session handoffs, and supports fast reconnection for roaming devices.
- c. The AP is assumed to have a secure, trusted link to the AS.

3) **Authentication Server (AS)**

- a. The AS is responsible for verifying the identity of IoT devices and granting network access.
- b. It performs mutual authentication, key distribution, and session management.

4) **Communication Assumptions**

- a. The link between the IoT Device (D) and the Access Point (AP) is considered insecure and vulnerable to eavesdropping, modification, and replay attacks.

- b. The connection between the Access Point (AP) and the Authentication Server (AS) is assumed to be secure, typically implemented over a protected backbone or VPN.
- c. IoT devices cannot communicate directly with the AS; all authentication and session establishment must be routed through the AP.

Framework Requirement: In the proposed framework, IoT devices establish initial connections to the access point via an untrusted, insecure wireless channel. It is architecturally assumed that a secure, trusted channel exists between the access point and the authentication server. IoT devices cannot establish direct communication with the authentication server; all communication must be relayed through the access point.

Consequently, the end-to-end communication path between the IoT device and the authentication server inherits the security vulnerabilities of the initial wireless link and is thus considered untrusted. This model necessitates that robust mutual authentication be performed between the device and the authentication server prior to granting network access. Only upon successful cryptographic verification of each other's legitimacy will the authentication server authorise the access point to permit the device to connect to the network. It assumes that the Authentication Server's private key remains uncompromised.

3.3. Threat model

To rigorously evaluate the security of the proposed protocol, we adopt two well-established adversarial frameworks: the Dolev-Yao (DY) model [28] and the Canetti-Krawczyk (CK) adversary model [29]. These models assume all communication occurs over an insecure public channel.

The Dolev-Yao (DY) model characterizes an adversary with complete control over the communication channel, granting the capabilities to eavesdrop on, intercept, modify, delete, replay, or inject fabricated messages. The CK-adversary model extends these capabilities further. In addition to the DY powers, a CK adversary can:

- i. Target specific protocol sessions.
- ii. Compromise certain types of secret information through defined queries, such as exposing long-term keys, ephemeral (temporary) secrets, or the internal session state of a particular instance.

Furthermore, both models consider the threat of physical device capture, where an adversary obtains a legitimate IoT device and can attempt to extract stored secrets or manipulate its operation.

Synthesizing these models, we define the following adversarial capabilities within our WLAN-IoT environment:

1. **Complete Channel Control:** The adversary (A) can eavesdrop on, intercept, modify, replay, or fabricate any message transmitted over the wireless channel between the IoT device and the Access Point.
2. **Offline Guessing:** A can perform offline, polynomial-time guessing attacks on weak credentials (e.g., low-entropy passwords or predictable identifiers).
3. **Session Analysis & Traceability:** By passively capturing multiple protocol sessions, A can attempt to analyse message patterns to link different sessions to the same device, thereby compromising user privacy.
4. **Man-in-the-Middle (MITM) Position:** A can actively interpose itself between communicating parties, impersonating the Device (D) to the Access Point (AP)/Authentication Server (AS), and vice-versa, during the authentication exchange.
5. **Physical Device Compromise:** A can physically capture an IoT device and perform attacks such as extracting stored data from memory, probing side-channels, or tampering with its firmware to recover long-term secrets or manipulate its cryptographic operations.

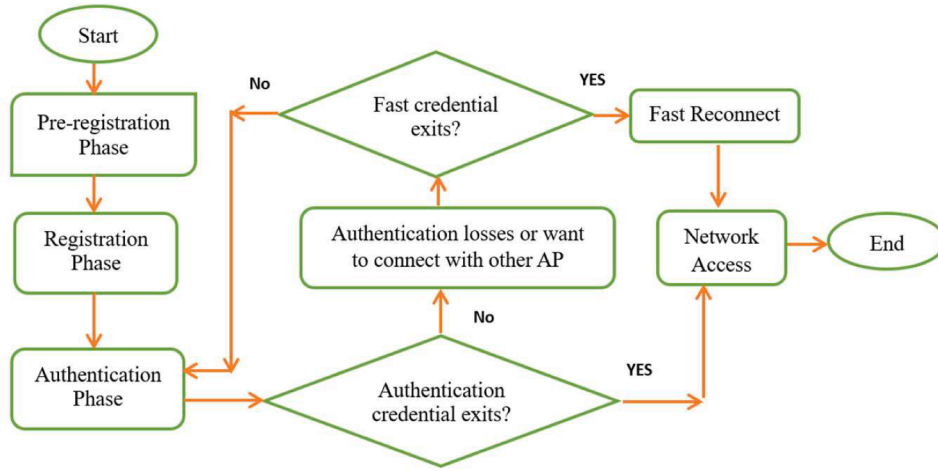


Fig. 2. flowchart of the proposed protocol.

This comprehensive threat model ensures the protocol is designed to be resilient against both network-based and physical attacks, providing robust security guarantees for the WLAN-IoT framework.

3.4. System initialization and parameter specification

3.4.1. System initialization

Before deployment, the following system-wide parameters and keys are established:

Cryptographic Primitives: A secure cryptographic hash function $h(\cdot)$ (e.g., SHA-256), a symmetric encryption algorithm (e.g., AES-256 in CBC mode), and an elliptic curve domain for ECC are selected and agreed upon.

Authentication Server (AS) Setup: The AS generates its long-term public-private key pair (PKS, SKS) for ECC. It also establishes a shared symmetric secret APS with every legitimate Access Point (AP) in the network.

Device Manufacturer Setup: The manufacturer embeds a Physical Unclonable Function (PUF) and a unique identity Udd into each IoT device. A preliminary manufacturer password is also set.

Parameter Constants: System-wide constants are defined, including:

- i) The maximum allowed clock skew ΔT .
- ii) The DoS attempt threshold $DosCount$.
- iii) The default lease time LT and fast-reconnect expiry time ET^{FR} .
- iv) The size of the one-time nonce list $(|OT_{list}|, \text{e.g., } 100 \text{ entries})$ and the PUF challenge-response pool $(|P_{list}|, \text{e.g., } 20 \text{ entries})$.

3.4.2. Significance of core cryptographic parameters

The role and significance of key dynamic parameters generated during the protocol are clarified:

- 1) Master Symmetric Key (MSK): $MSK = h(Na || SKS || I_i)$ This is a session-independent, long-term secret derived from the server's private key and the device's pseudo-identity. It serves as a root key for generating masked identities $(MSId_i)$ and is never transmitted over the network. Its compromise would require breaching the server's private key.
- 2) Registration Symmetric Key (K_{reg}): $K_{reg} = h(Na \oplus Nb \oplus I_i)$ This is a medium-term secret established during the registration phase. It is used to encrypt the initial payload containing the OT_{list} and the MSK for secure transfer to the device. It is subsequently updated (to K^{new}) after each successful authentication.
- 3) Session Key (SK): $SK = h(K^{new} || N_d || I_u^N)$ This is a short-term, ephemeral key derived from fresh nonces and the updated

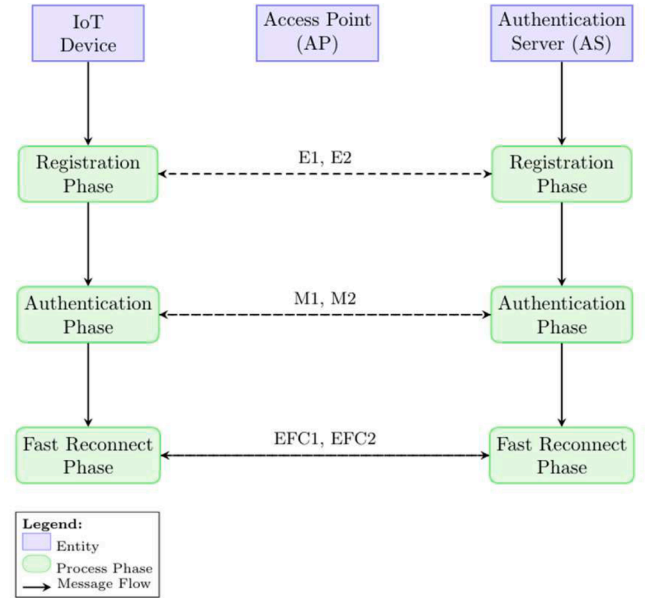


Fig. 3. Overall protocol architecture showing three entities and three phases.

symmetric key. It provides perfect forward secrecy for data encryption within a single session, as it depends on one-time nonces (OT_j, N_c, N_d, R_i) that are deleted after use.

- 4) Key Update Principle: The protocol employs a key hierarchy and continuous updating strategy. The long-term MSK remains static (bound to the device's identity and server's key). The K_{reg}/K^{new} key is updated with every authentication, and the session key SK is fresh for every session. This design limits the impact of any potential key compromise.

4. Proposed protocol

This section proposed an improved IoT-WLAN authentication protocol to address the limitations and security flaws of the current state-of-the-art authentication protocols. The EAP-based protocols have three participants: Internet of Things devices, authentication servers, and access points. The proposed protocol consists of three phases: registration, authentication, and the fast reconnect phase. Fig. 2 shows the flowchart of the proposed protocol. Fig. 3 demonstrate the overall protocol architecture showing three entities and three phases.

Table 1
Notations and meaning.

Notation	Meaning
UID_d	Unique Identity of IoT device
Sid_i	Server Identity
PSW	Password stored in tamper-resistant storage
$PUF(C)$	Physical Unclonable Function response to challenge C
R, W	Reliable secret and helper data from Fuzzy Extractor
PUF_{Secret}	$h(R UID_d)$ where h is a cryptographic hash function
$Combined_{Secret}$	$h(PSW PUF_{Secret})$
I_i	Device identity: $h(Combined_{Secret} \oplus UID_d)$
N_a, N_b, N_c, N_d	Nonces (128-bit random numbers)
MSK	Master Symmetric Key: $h(N_a SKS I_i)$
Mid_i	Masked device identity: $ENC_{MSK}(I_i r_i)$
$MSid_i$	Masked server identity: $ENC_{MSK}(Sid_i r_i)$
$MTid_i^T$	Masked Temporary Identity for Fast Reconnect
OT_j	One-time nonce for session j (128-bit)
OT_j^{index}	Index of the used OT_j in OT_list
OT_list	List of One-Time Pre-Nonces for Perfect Forward Secrecy
I_i	Temporary device identity: $h(I_i \oplus R_1)$
I_i^N	New temporary identity: $h(I_i \oplus R_1 \oplus r_2)$
K_{new}	New symmetric key: $h(K_{temp} N_d r_2)$
K_{temp}	Temporary session key: $h(MSK OT_j N_c R_1)$
SK	Final session key: $h(K_{new} I_i^N)$
TSK	Temporary secret key for fast reconnect
ΔT	Maximum allowed time difference (message freshness)
APS	Symmetric shared secret key between Service provider and AP
SKS, PKS	Server's private and public keys
$S_p, SI_i, C_p, FC_p, FSI_i, SC_i$	DoS counters
$DoSCount$	DoS attempt threshold
$FuzzyExtractor(.)$	Function to extract reliable secret from noisy PUF
$FuzzyExtractorRecover(.)$	Function to recover secret using PUF and helper data
FER	FuzzyExtractorRecover
LT	Lease Time for fast reconnect
R^2, R^3	Random numbers in fast reconnect phase
SK^T	Temporary session key for fast reconnect
$h()$	Cryptographic hash function
$ENC_{Key}(.), DK_{Key}(.)$	Encryption, Decryption with Key
$ $	Concatenation
$\oplus$	XOR
$Stored_Blob$	Encrypted container of device credentials

- 1) Pre-registration Phase: The devices get the temporary credentials like temporary identities and expiry time from authentication server. In this phase device get the secure password from authentication server which is the combination of manufacturer password and service provider password which will stored in temper-resistance hardware.
- 2) Registration Phase: The authentication server and devices exchange the security credentials for authentication in a public channel.

- 3) Authentication Phase: The devices and authentication server mutually authenticate to each other and after mutual authentication, both parties exchange the temporary security credentials for the generation of session keys. This phase preserves the confidentiality and integrity of data.

Fast-reconnect phase: This phase facilitates the device to fast reconnect with the access point by quick reauthentication with access point whenever disconnection will happen due to a network problem.

We follow the same assumption as research article [30], Lee et al. [31] that the clocks of D, AP, and AS are synchronised. Table 1 represents the notation and abbreviations used in the paper.

4.1. Pre- registration phase

The pre-registration phase establishes secure device credentials through dual-control provisioning between the manufacturer and service provider. The manufacturer creates a temporary identity of devices as Tid_i and expiry time ET protected with a password generated by the manufacturer, which is stored in tamper-resistant hardware [32]. Then the shipment of the device comes with a chain of custody to the service provider. The service provider verifies the manufacturer's credentials and request for ownership of this device. After obtaining ownership of the device, the service provider generates its own device password and computes the final device password, which combines both the manufacturer's password and the service provider's password; this final password is then stored in tamper-proof hardware. This password is ready to securely store the device credentials. The service provider stores the service provider-generated password. The device may store the manufactured password encrypted with the dual control final security password in tamper-proof hardware. The service provider generates the temporary device identity Tid_i and expiry time ET . In this method the device's password cannot be compromised even if the manufacturer-generated and server-generated passwords are compromised. Then the service provider renews the pre-registration security credentials for the registration phase, which is ready for the registration process for the devices. This phase secures the perfect forward security along with insider attacks in the future. The resultant PSW is a device-specific cryptographic seed, derived from both the manufacturer and service provider, and is permanently stored within the device's tamper-resistant hardware. It is not intended for user input but acts as a root secret for the device's identity and key generation. Fig. 4 shows the phases of pre-registration phase.

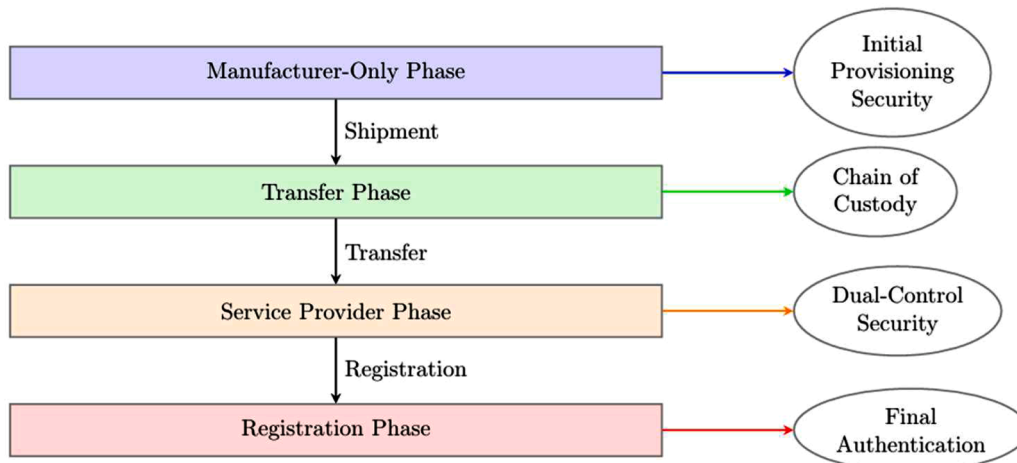


Fig. 4. Pre-registration phase for generation of device password.

Algorithm 1

(Device Registration Initialization).

Input: Uid_d, PSW

Output: E_1, H_1

Begin:

- 1: $PSW \leftarrow \text{StoreInTamperResistantMemory}(PSW)$
- 2: $C \leftarrow h(Uid_d || \text{"PUF CHALLENGE"})$
- 3: $(R, W) \leftarrow \text{FuzzyExtractor}(PUF(Challenge))$
- 4: $PUF_{Secret} \leftarrow h(R || Uid_d)$
- 5: $Combined_{Secret} \leftarrow h(PSW || PUF_{Secret})$
- 6: $I_i \leftarrow h(Combined_{Secret} \oplus Uid_d)$
- 7: $N_a \leftarrow \text{GenerateRandom}(128)$
- 8: $T_1 \leftarrow \text{CurrentTimestamp}()$
- 9: $ET \leftarrow \text{SetExpiryTime}()$
- 10: $LIA_i \leftarrow \text{GetLocationID}()$
- 11: $TId_i \leftarrow \text{GetTempID}()$
- 12: **for** $m = 1$ to 20 **do**
- 13: $P_{list}[m] \leftarrow (R^m, C^m) \leftarrow \text{FuzzyExtractor}(PUF(C^m))$
- 14: **end for**
- 15: $E_1 \leftarrow ENC_{PKS}(I_i || N_a || T_1 || ET || LIA_i || TId_i || W || P_{list}[m])$
- 16: $H_1 \leftarrow h(I_i || N_a || T_1 || ET || LIA_i || TId_i || W || P_{list}[m])$
- 17: **return** (E_1, H_1, TId_i)

End

Algorithm 2

(Server Registration Processing).

Input: (E_1, H_1, TId_i)

Output: E_2, H_2

Begin:

- 1: **if** TId_i valid **then**
- 2: **if** TId_i is at first time for authentication **then**
- 3: $C_i \leftarrow 0$ // Initialize DoS counter
- 4: **end if**
- 5: **else**
- 6: **abort** "Identity check failed" and $FC_i \leftarrow FC_i + 1$
- 7: **end if**
- 8: $(I_b, N_a, T_1, ET, LIA_i, TId_i, W, P_{list}[m]) \leftarrow DK_{SKS}(E_1)$
- 9: **if** $h(I_i || N_a || T_1 || ET || LIA_i || TId_i || W || P_{list}[m]) \neq H_1$ **then**
- 10: **abort** "Integrity check failed", $C_i \leftarrow C_i + 1$
- 11: **end if**
- 12: **if** $T_2 \leftarrow \text{CurrentTime}() - T_1 < \Delta T$, $C_i > DOS_Threshold$ and ET, LIA_i are not valid **then**
- 13: **abort** "Message expired"
- 14: **end if**
- 15: $MSK \leftarrow h(N_a || SKS || I_i)$
- 16: $r_1 \leftarrow \text{GenerateRandom}(128)$
- 17: $Mid_i \leftarrow ENC_{MSK}(I_i || r_1)$
- 18: $MSId_i \leftarrow ENC_{MSK}(Sid_i || r_1)$
- 19: $N_b \leftarrow \text{GenerateRandom}(128)$
- 20: $K_{reg} \leftarrow h(N_a \oplus N_b \oplus I_i)$
- 21: **for** $j = 1$ to 100 **do**
- 22: $OT_{list}[j] \leftarrow \text{GenerateRandom}(128)$
- 23: **end for**
- 24: $OT_{enc} \leftarrow ENC_{Kreg}(OT_{list})$
- 25: $MSK_{enc} \leftarrow ENC_{Kreg}(MSK)$
- 26: $T_3 \leftarrow \text{CurrentTimestamp}()$
- 27: $E_2 \leftarrow ENC_{Na \oplus I_i}(Mid_i || MSId_i || N_b || T_3 || MSK_{enc} || OT_{enc})$
- 28: $H_2 \leftarrow h(Mid_i || MSId_i || N_b || T_3 || MSK_{enc} || OT_{enc})$
- 29: **Store in database:** $(I_b, Mid_i, MSId_i, MSK, LIA_i, K_{reg}, OT_{list}, r_1, W, P_{list}[m])$
- 30: **return** (E_2, H_2)

End

4.2. Registration phase

In this phase the Internet of Things device (D) registers itself first to the authentication server (AS). Algorithms 1, 2, and 3 describe the process of registration of a device. Algorithm 1 describes the device registration initialisation; Algorithm 2 describes the device registration process on the server side. Algorithm 3 device storage setup. Fig. 5 depicted the registration phase. This phase occurs over an insecure channel, and the subsequent steps are performed during this phase as follows:

1. **Algorithm 1** (Device Registration Initialization): Device D extract the password PSW from securely stored in the dual control password in temper resistant hardware which is the root secret established during pre-registration phase. Device generate a standard string as "PUF CHALLENGE" then compute enrolment $Challenge C = h(Uid_d || \text{"PUF CHALLENGE"})$ by hashing with unique identity of device which creates a deterministic device specific challenges. Then PUF hardware generate the PUF response with public helper data 'W' with input as $Challenge$ like $(R, W) \leftarrow \text{FuzzyExtractor}(PUF(C))$. Devices derive a device-specific PUF secret by hashing the extracted secret R with the device unique identity Uid_d as $PUF_{Secret} \leftarrow h(R || Uid_d)$. This

Algorithm 3

(Device Storage Setup).

Input: $\langle E_2, H_2 \rangle$ from Server

Output: Stored credentials in device memory

Begin:

- 1: $(Mid_i, MSId_i, N_b, T_3, MSK_{enc}, OT_{enc}) \leftarrow DEC_{Na \oplus I_i}(E_2)$
- 2: if $T_4 \leftarrow \text{CurrentTimestamp}() - T_3 < \Delta T$, and then
- 3: if $h(Mid_i || MSId_i || N_b || T_3 || MSK_{enc} || OT_{enc}) \neq H_2$ then
- 4: abort ("Server response integrity failed ") and $FSI_i = FSI_i + 1$
- 5: end if
- 6: else
- 7: $K_{reg} \leftarrow h(N_a \oplus N_b \oplus I_i)$
- 8: $OT_{list} \leftarrow DKC_{K_{reg}}(OT_{enc})$
- 9: $SI_i \leftarrow 0$ // Initialize DoS counter
- 10: Store $Stored_Blob = ENC_{CombinedSecret}(UId_d, Mid_i, MSId_i, LIA_i, ET, SI_i=0, OT_{list})$ and C, W, UId_d .
- 11: STORE_IN_FLASH($Stored_Blob$)
- 12: end if
- 13: Return Success

End

binds the PUF secret to the specific device. Then device creates a combined secret by hashing the stored password and the PUF secrets. This combined secret becomes the master secret for the device which is combination of password knowledge and physical hardware possession. Then device computes pseudonymous device identity $I_i \leftarrow h(Combined_{Secret} \oplus UId_d)$ which will be used as the device identifier with server. Then device generates the Nonce N_a and select current time stamp T_1 . Device extract ET and TId_i then get the location identity LIA_i . Device creates a pool of pre-computed PUF challenge-response pairs $P_{list}[m] \leftarrow (R^m, C^m) \leftarrow \text{FuzzyExtractor}(PUF(C^m))$. Then device compute the encrypted credentials with service provider public key $E_1 = ENC_{PKS}(I_i || N_a || T_1 || ET || LIA_i || TId_i || W || P_{list}[m])$ and its hash as: $H_1 = h(I_i || N_a || T_1 || ET || LIA_i || TId_i || W || P_{list}[m])$. Outputs the encrypted E_1 , its integrity hash H_1 and TId_i are send to authentication server via access point.

2. **Algorithm 2** (Server Registration Processing): Upon receiving of $\langle E_1, H_1, TId_i \rangle$ by AS, first checks the TId_i is valid by matching with own device database. if this TId_i not present in database then abort the communications and the increment the DoS counter as $FC_i \leftarrow FC_i + 1$ with marking the device transmission credentials. if the device comes for first time for registration, then initialize the DoS counter $C_i = 0$. Then authentication server decrypts the E_1 with own private key SKS and get the credentials $\langle I_i, N_a, T_1, ET, LIA_i, TId_i, W, P_{list}[m] \rangle \leftarrow DK_{SKS}(E_1)$. The AS will compute the hash value with decrypted credentials $H_1^* = h(I_i || N_a || T_1 || ET || LIA_i || TId_i || W || P_{list}[m])$ and match with received H_1 . If integrity checked failed $H_1^* \neq H_1$ then abort the authentication and increment the DoS counter $C_i = C_i + 1$. AS then check $T_2 \leftarrow \text{CurrentTime}() - T_1 < \Delta T$ is not valid abort the communication and marked as message is expired. AS checks the $C_i > DOS_Threshold$, ET and LIA_i . If these credentials valid than calculate the master secret key $MSK = h(N_a || SKS || I_i)$. AS generates random number r_1 and compute pseudo identity of device and AS $Mid_i = ENC_{MSK}(I_i || r_1)$ and $MSId_i = ENC_{MSK}(Si_d || r_1)$ respectively. Then AS generate Nonce N_b and compute the symmetric registration secret key $K_{reg} = h(N_a \oplus N_b \oplus I_i)$. AS generates the one-time pre-nonces list OT_{list} to achieve perfect forward secrecy. The MSK and OT_{list} are encrypted with K_{reg} produces OT_{enc} and MSK_{enc} . AS select the current timestamp T_3 and compute $E_2 = ENC_{Na \oplus I_i}(Mid_i || MSId_i || N_b || T_3 || MSK_{enc} || OT_{enc})$ and $H_2 = h(Mid_i || MSId_i || N_b || T_3 || MSK_{enc} || OT_{enc})$. AS stores securely the credentials $\langle I_i, Mid_i, MSId_i, MSK, LIA_i, K_{reg}, OT_{list}, r_1, W, P_{list}[m] \rangle$ in own database. AS sends $\langle E_2, H_2 \rangle$ to device.
3. **Algorithm 3** (Device Storage Setup): Upon receiving of $\langle E_2, H_2 \rangle$ from AS, the device checks the time duration between first message and received message if its within time then its decrypt the E_2 as $DEC_{Na \oplus I_i}(E_2)$ and get the credentials $Mid_i, MSId_i, N_b, T_3, MSK_{enc}, OT_{enc}$. Device also checks the $T_4 \leftarrow \text{CurrentTimestamp}() - T_3 < \Delta T$, if it fails then marks as message is expired and abort the communications and

registrations process is failed and reinitiates the registration process and increment the DoS counter of registration process $FSI_i = FSI_i + 1$. if pass the condition then device calculate the $K_{reg} = h(N_a \oplus N_b \oplus I_i)$ and decrypt the one-time pre-nonces list $OT_{list} \leftarrow DKC_{K_{reg}}(OT_{enc})$. Initialise the DoS counter for the authentication process as $SI_i \leftarrow 0$ and then store the all credentials encrypted as $ENC_{CombinedSecret}(UId_d, Mid_i, MSId_i, LIA_i, ET, SI_i=0, OT_{list}, W)$ and C, UId_d .

4.2. Authentication phase

In this phase device D want to connect and communicate with and the authentication server. For connection with AS, both device and AS must be mutually authenticate to each other. In the authentication process device uses the security credentials which is exchanged in registration phase. **Algorithm 4** describes the device authentication request. **Algorithm 5** describes the Authentication server process of device request and authentication of device and it successful then session key and fast reconnect credentials is generated for further communication. **Algorithm 6** describes the server authentication by devices if its successful device and AS both are authenticated and accept the session key and fast reconnect credentials. **Fig. 6** depict the authentication phase pictorially.

Algorithms of Authentication phase are described as flows:

- 1) (**Algorithm 4**): Whenever IoT device wants to communicate with authentication server it will first connect with nearby AP. If it is first time connection with AP or authentication credentials is expired due to disconnection or roaming with other AP where fast credentials does not exists then device initiates the authentication process as device authentication request. In that request the device first fetch the PSW from temper resistant hardware. Then device extract the Challenge C, W and UId_d and compute $R' \leftarrow FER(PUF(C), W)$. Then device. compute $PUF_{Secret} \leftarrow h(R || UId_d)$, $Combined_{Secret} \leftarrow h(PSW || PUF_{Secret})$ and $Combined_{Secret} \leftarrow h(PSW || PUF_{Secret})$. Then now device able to decrypt the stored credentials as $DK_{CombinedSecret}(StoredBlob)$ and get $UId_d, Mid_i, MSId_i, LIA_i, ET, SI_i=0, OT_{list}$. Then checks the $SI_i \geq DosCount$ if it is true then DoS threshold exceeded and new credentials of registration phase are required. It happens only when earlier DoS attacked or multiple times authentication failures are happened. In this situation device must go to registration phase again as per policy. If $SI_i \leq DosCount$ then calculate device pseudo identity $I_i \leftarrow h(Combined_{Secret} \oplus UId_d)$. Device select unused one-time pre-nonces from OT_{list} and marked as OT_{index} and its value as OT_j . Device generates the random number R_1 and Nonce N_c . Device select the current time stamp T^1 and compute $I_u \leftarrow h(I_i \oplus R_1)$. Then device generate the message $M1 = ENC_{K_{reg}}(I_u || MSId_i || N_c || T^1 || R1 ||$

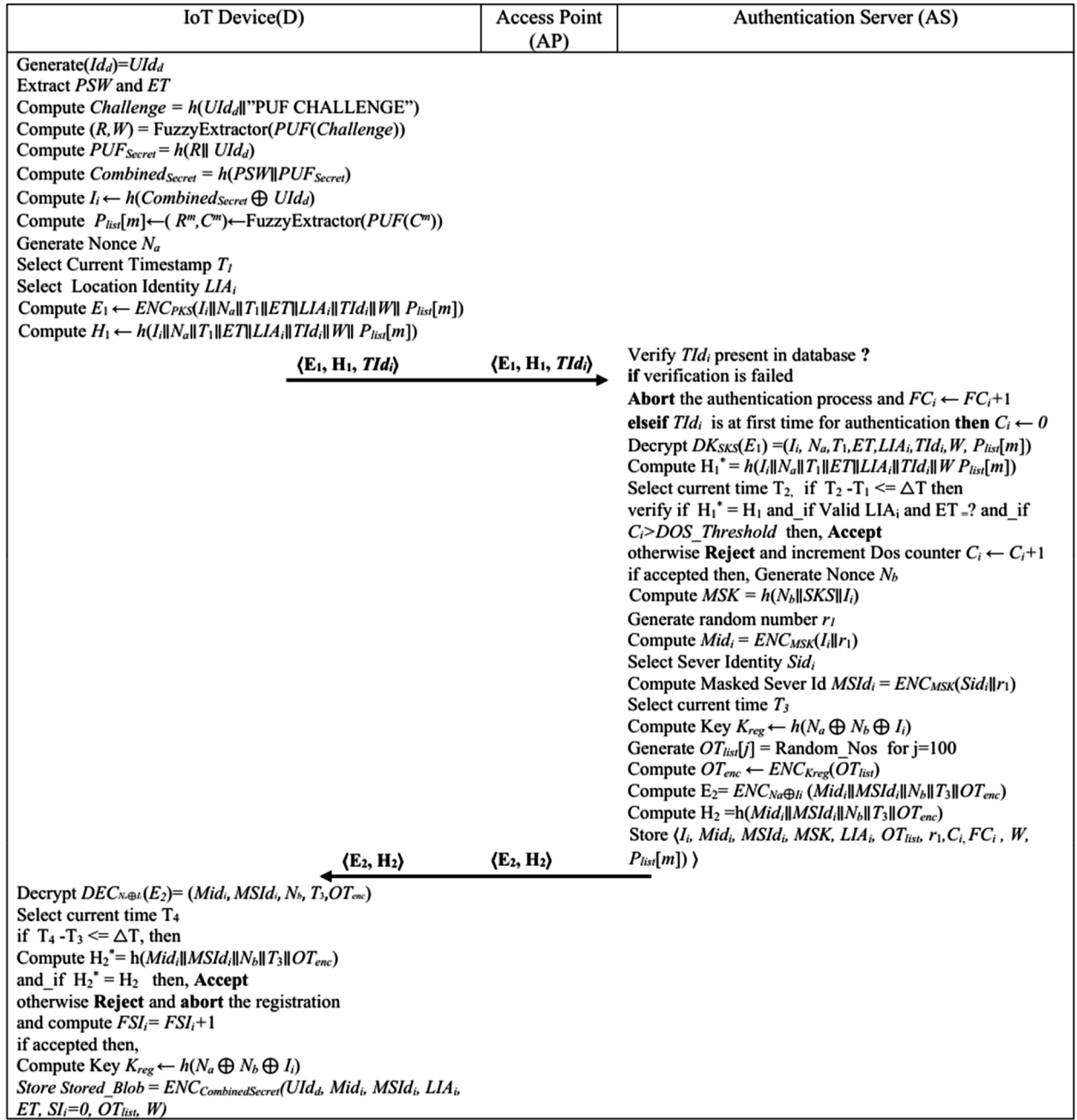


Fig. 5. Registration phase.

$LIA_i || OT_{index}$) and hash $HA_1 = h(I_u || MSId_i || N_c || T^1 || R_1 || LIA_i || OT_{index})$. Device send the message $\langle Mid_i, M_1, HA_1, T^1 \rangle$ to AS via AP.

- 2) (Algorithm 5): After receiving the message by authentication server AS first select the current time stamp T^2 , if $T^2 - T^1 > \Delta T$ then abort the authentication process marked as message is expired and increment the DoS counter as $SC_i \leftarrow SC_i + 1$. If message not expired then search the device credentials with Mid_i and fetch the credentials from database $I_i, MSK, OT_{list}, MSId_i, r_1, S_b, W$. Then AS decrypt the Mid_i and $DK_{MSK}(Mid_i)$ and get the I_i, r_1 . If $I_i = I_i$ and $r_1 = r_1$ then decrypt the message

$DK_{Kreg}(M_1)$ and get $I_u, MSId_i, N_c, T^1, R_1, LIA_i, OT_{index}$ and calculate $HA_1^* = h(I_u || MSId_i || N_c || T^1 || R_1 || LIA_i || OT_{index})$. if $HA_1^* \neq HA_1$ or $MSId_i \neq MSId_i$ or $I_u \neq I_u$ or $R_1 = R_1$ then due to integrity check is failed, abort the authentication process and increment the DoS counter as $S_i \leftarrow S_i + 1$. If integrity check is passed then device is authenticated by

authentication server. After that select the one-time pre-nonces OT_j from list with OT_{index} then marked as used as one-time pre-nonces. Afterward AS generates random number r_2 and nonce N_d then select the current timestamp T^3 . AS compute the new key $K^{new} \leftarrow h(K_{reg} || OT_j || N_c || R_1)$ and new device identity $I_u^{new} \leftarrow h(I_i \oplus R_1 \oplus r_2)$. After that AS replace the key as $K_{reg} \rightarrow K^{new}$. AS then compute the session key $SK \leftarrow h(K^{new} || N_d || I_u^{new})$. AS compute new masked device and server as identity $Mid_i^{new} \leftarrow ENC_{MSK}(I_i || r_2)$ and $MSId_i^{new} \leftarrow ENC_{MSK}(Sid_i || r_2)$. AS then generate fast reconnect credential like secret key TSK set fast reconnect expiry time ET^{FR} . AS select a PUF credential as $P_{index} = (R^m, C^m) \leftarrow R^m, C^m$ and marked as used P_{index} . AS compute new masked fast reconnect device identity encrypted with shared secret symmetric key of AP as $MTId_i^T = ENC_{APS}(Mid_i^{new})$. AS now create an encrypted message $M_2 \leftarrow ENC_{Knew}(I_u^{new} || Mid_i^{new} || MSId_i^{new} || N_d || T^3 || r_2 || TSK || ET^{FR} || MTId_i^T)$ and hash $HA_2 \leftarrow h(I_u^{new} || Mid_i^{new} || MSId_i^{new} || N_d || T^3 || r_2 || TSK || ET^{FR} || MTId_i^T)$.

Algorithm 4Device authentication request (M_1, HA_1).

Input: Authentication Request by device

Output: $\langle Mid_b, M_1, HA_1, T^1 \rangle$

Begin:

- 1: $PSW \leftarrow \text{StoreInTamperResistantMemory}(PSW)$
- 2: $R' \leftarrow \text{FER}(PUF(C), W)$
- 3: $PUF_{Secret} \leftarrow h(R || Uld_d)$
- 4: $Combined_{Secret} \leftarrow h(PSW || PUF_{Secret})$
- 5: $(Uld_d, Mid_b, MSId_b, LIA_b, ET, SI_i=0, OT_{list}) \leftarrow DK_{CombinedSecret}(Stored_Blob)$
- 6: **if** $SI_i \geq DoSCount$ **then**
- 7: **abort** "DoS threshold exceeded"
- 8: **end if**
- 9: $I_i \leftarrow h(Combined_{Secret} \oplus Uld_d)$
- 10: $OT_j, OT_{index} \leftarrow \text{SelectUnusedOT}(OT_{list})$
- 11: $\text{MarkAsUsed}(OT_{index})$
- 12: $R_1 \leftarrow \text{GenerateRandom}(128)$
- 13: $N_c \leftarrow \text{GenerateRandom}(128)$
- 14: $T^1 \leftarrow \text{CurrentTimestamp}()$
- 15: $I_u \leftarrow h(I_i \oplus R_1)$
- 16: $M_1 \leftarrow ENC_{K_{reg}}(I_u || MSId_i || N_c || T^1 || R_1 || LIA_i || OT_{index})$
- 17: $HA_1 \leftarrow h(I_u || MSId_i || N_c || T^1 || R_1 || LIA_i || OT_{index})$
- 18: **return** $\langle Mid_b, M_1, HA_1, T^1 \rangle$

End

Algorithm 5Device authentication by server (M_2, HA_2).

Input: $\langle Mid_b, M_1, HA_1, T^1 \rangle$

Output: $\langle M_2, HA_2, MTId_i^T, ET^{FR}, TSK, T^3 \rangle$

Begin:

- 1: **if** $T^2 \leftarrow \text{CurrentTimestamp}() - T^1 > \Delta T$ **then**
- 2: **abort** "Message expired"
- 3: $SC_i \leftarrow SC_i + 1$
- 4: **else**
- 5: $(I_b, MSK, OT_{list}, MSId_b, r_1, S_b, W) \leftarrow \text{LookupByMid}_i(Mid_i)$ and compute $(I_i', r_1') \leftarrow DK_{MSK}(Mid_i)$
- 6: **if** $I_i = I_i'$ and $r_1 = r_1'$ **then**
- 7: $(I_u, MSId_i', N_c, T^1, R_1, LIA_i, OT_{index}) \leftarrow DK_{K_{reg}}(M_1)$
- 8: **if** $h(I_u || MSId_i' || N_c || T^1 || R_1 || LIA_i || OT_{index}) \neq HA_1$, or $MSId_i' \neq MSId_b$, or $I_u \neq I_u$ or $R_1 = R_1$ **then**
- 9: **abort** "Integrity check failed"
- 10: $S_i \leftarrow S_i + 1$
- 11: **end if**
- 12: **else**
- 13: IoT device is **Authenticated** by AS
- 14: $OT_j \leftarrow OT_{list}[OT_{index}]$
- 15: $\text{MarkOTUsed}(OT_{index})$
- 16: $r_2 \leftarrow \text{GenerateRandom}(128)$
- 17: $N_d \leftarrow \text{GenerateRandom}(128)$
- 18: $T^3 \leftarrow \text{CurrentTimestamp}()$
- 19: $K^{new} \leftarrow h(K_{reg} || OT_j || N_c || R_1)$
- 20: $I_u^N \leftarrow h(I_i \oplus R_1 \oplus r_2)$
- 21: $\text{Update } K_{reg} \rightarrow K^{new}$
- 22: $SK \leftarrow h(K^{new} || N_d || I_u^N)$
- 23: $Mid_i^{new} \leftarrow ENC_{MSK}(I_i || r_2)$
- 24: $MSId_i^{new} \leftarrow ENC_{MSK}(MSId_i || r_2)$
- 25: $TSK \leftarrow \text{GenerateRandom}(128)$
- 26: $ET^{FR} \leftarrow \text{SetFastReconnectExpiry}()$
- 27: Select a $P_{index} = (R^n, C^n) \leftarrow (R^m, C^m)$, marked as used P_{index}
- 28: $MTId_i^T \leftarrow ENC_{APS}(Mid_i^{new})$
- 29: $M_2 \leftarrow ENCK_{new}(I_u^N || Mid_i^{new} || MSId_i^{new} || N_d || T^3 || r_2 || TSK || ET^{FR} || MTId_i^T)$
- 30: $HA_2 \leftarrow h(I_u^N || Mid_i^{new} || MSId_i^{new} || N_d || T^3 || r_2 || TSK || ET^{FR} || MTId_i^T)$
- 31: **Update database:** $Mid_i \rightarrow Mid_i^{new}, MSId_i \rightarrow MSId_i^{new}, r_1 \rightarrow r_2$
- 32: **return** $\langle M_2, HA_2, MTId_i^T, P_{index}, ET^{FR}, TSK, T^3 \rangle$
- 33: **end if**
- 34: **end if**

End

- Then AS update the credentials $Mid_i \rightarrow Mid_i^{new}$, $MSId_i \rightarrow MSId_i^{new}$ and $r_1 \rightarrow r_2$. The message $\langle M_2, HA_2, MTId_i^T, P_{index}, ET^{FR}, TSK, T^3 \rangle$ send to AP. AP extract the credentials of fast reconnect for device $\langle MTId_i^T, P_{index}, ET^{FR}, TSK, T^3 \rangle$ and message $\langle M_2, HA_2, MTId_i^T \rangle$ send to device.
- 3) (**Algorithm 6**): Upon receiving the message $\langle M_2, HA_2, MTId_i^T \rangle$ from AP, device select the current time stamp T^4 and checks if $T^4 - T^3 > \Delta T$, or $SI_i > DoS_count$ then abort the authentication process due to

response is expired and increment the DoS counter $SI_i = SI_i + 1$. If pass the checks then compute $K^{new} \leftarrow h(K_{reg} || OT_j || N_c || R_1)$ and decrypt the M_2 with key K^{new} and get the credentials as $I_u^N, Mid_i^{new}, MSId_i^{new}, N_d, T^3, r_2, TSK, ET^{FR}, MTId_i^T$. Then compute $HA_2^* = h(I_u^N || Mid_i^{new} || MSId_i^{new} || N_d || T^3 || r_2 || TSK || ET^{FR} || MTId_i^T)$ and $I_u^N \leftarrow h(I_i \oplus R_1 \oplus r_2)$. if $HA_2^* \neq HA_2$ or $I_u^N \neq I_u^N$, then integrity checked is failed and abort the authentication process and increment the DoS counter $SI_i =$

Algorithm 6

(Server authentication by device).

Input: $\langle M_2, HA_2, T^3 \rangle$ from Server

Output: Server Authentication by Device and session key (SK) generation, Updated credentials.

Begin:

```

1:  $T^4 \leftarrow \text{CurrentTimestamp}()$ 
2:  $T^4 - T^3 > \Delta T$ , or  $SI_i > \text{Dos\_count}$  then
3: abort "Response expired"
4:  $SI_i \leftarrow SI_i + 1$ 
5: end if
6:  $K^{new} \leftarrow h(K_{reg} || OT_i || N_c || R_1)$ 
7:  $I_u^N, Mid_i^{new}, MSId_i^{new}, Nd, T^3, r_2, TSK, ET^{FR}, MTId_i^T \leftarrow DK_{K^{new}}(M_2)$ 
8: if  $h(I_u^N || Mid_i^{new} || MSId_i^{new} || Nd || T^3 || TSK || ET^{FR} || MTId_i^T)$ 
    $\neq HA_2$  then
9: abort "Integrity check failed"
10:  $SI_i \leftarrow SI_i + 1$ 
11: else
12:    $I_u^N \leftarrow h(I_i \oplus R_1 \oplus r_2)$ 
13:   if  $I_u^N \neq I_u^N$  then
14:      $SI_i \leftarrow SI_i + 1$ 
15:   abort "Server authentication failed"
16:   else "Server authenticated successfully"
17:    $SK \leftarrow h(K^{new} || Nd || I_u^N)$ 
18:   Update  $Mid_i \rightarrow Mid_i^{new}, MSId_i \rightarrow MSId_i^{new}$ 
19:   Update  $\text{Stored\_Blob}(UID_b, Mid_i^{new}, MSId_i^{new}, K^{new}, K_{reg}^P, LIA_b, ET, SI_b, OT_{list}, W, TSK, MTId_i^T, ET^{FR})$ 
20:   Remove used  $OT_{index}$ 
21: end if
22: end if
23: End

```

SI_i+1 . If this integrity checks are passed then authentication server AS is authenticated by device, thus both AS and device mutually authenticated each other. After successful authentication device compute session key $SK \leftarrow h(K^{new} || Nd || I_u^N)$ and update the credentials as $Mid_i \rightarrow Mid_i^{new}, MSId_i \rightarrow MSId_i^{new}$. After that device store the credential as $ENC_{CombinedSecret}(UID_b, Mid_i^{new}, MSId_i^{new}, K^{new}, K_{reg}^P, LIA_b, ET, SI_b, OT_{list}, W, TSK, MTId_i^T, ET^{FR})$ and remove the used OT_{index} .

4.3. Fast reconnect phase

The protocol supports efficient session recovery via a Fast Reconnection Phase. If a previously authenticated device DD is temporarily disconnected (e.g., due to network fluctuation or AP roam), it can rapidly reauthenticate using cached fast reconnect credentials, bypassing the full authentication protocol. This is permitted only within a predefined Expiry Time for Fast Reconnect (ET^{FR}). The ET^{FR} is configurable based on security policy commonly up to 24 h in enterprise settings, but adjustable for stricter or lighter requirements. The steps of this phase are detailed in Fig. 7.

- 1) Device D selects timestamp T^{F1} and generates random number R^2 . After that the device encrypts the fast reconnect credentials with the temporary secret key received in the authentication phase as $EFC^1 = ENC_{TSK}(Mid_i^{New} || R^2 || T^{F1})$ and computes the hash as $HT_1 = (Mid_i^{New} || R^2 || T^{F1})$. Device fetches the credentials from the database as $MTId_i^T$, then sends the message $\langle \{MTId_i^T, EFC^1, HT_1\} \rangle$ to AP.
- 2) D $\rightarrow$ AP: After receiving the message, it decrypts as $DK_{AP}(MTId_i^T)$ to get device identity Mid_i^{New} . If the device identity is present for fast reconnect, it proceeds for decrypting the EFC^1 to get the credentials $Mid_i^{New*}, R^{2*}, T^{F1}$. Select the present timestamp T^{F2} and if $T^{F2} - T^{F1} < \Delta T$ and valid ET^{FR} then compute $HT_1^* = (Mid_i^{New*} || R^{2*} || T^{F1})$ from credentials obtained from the decryption of EFC^1 . Verifying $HT_1^* = HT$, and $Mid_i^{New*} = Mid_i^{New}$ if verification is not successful, then abort the fast reconnect. If verification is successful then generate random number R^3 and lease time LT for reconnect. After that calculate the new device id as $MTId_i^{NEW} = h(R^3 \oplus MTId_i^T)$ and the temporary session key as $SK^T = h(TSK || R^3 || R^2 || R^n)$. Afterward encrypt the credentials as $EFC^2 = ENC_{TSK \oplus R^n}(MTId_i^{NEW} || R^2 || R^3 || T^{F3} || LT || C^n)$. and calculate the hash $HT_2 = h(MTId_i^{NEW} ||$

$R^2 || R^3 || T^{F3} || LT || C^n || R^n$. The message $\langle MTId_i^{NEW}, EFC^2, HT_2, C^n, T^{F3} \rangle$ sent to the device.

- 3) AP $\rightarrow$ D: Device D get the message $\langle MTId_i^{NEW}, EFC^2, HT_2, C^n, T^{F3} \rangle$ device select the current time stamp T^{F4} and checking the threshold time as $T^{F4} - T^{F3} < \Delta T$ if it is within time period then device will compute the PUF response as $R^n = FER(PUF(C^n), W)$. After that device calculate the encryption key as $TSK \oplus R^n$ and decrypt the EFC^2 extract the credentials $MTId_i^{NEW*}, R^{2*}, R^{3*}, T^{F3}, LT, C^n$. After that device compute $HT_2^* = h(MTId_i^{NEW*} || R^{2*} || R^{3*} || T^{F3} || LT || C^n || R^n)$ with extracted credentials.

Then verify if $HT_2^* = HT_2$, $MTId_i^{NEW*} = MTId_i^{NEW}$ and $R^{2*} = R^2$ then fast reconnect authentication get successful otherwise abort the fast reconnect phase. If authentication is successful compute fast reconnect session key $SK^T = h(TSK || R^3 || R^2 || R^n)$ and device is ready for transmission for secure messaging.

5. Informal security analysis

In this section we examined the protocol and informally demonstrated that this protocol is secure against the threat model to achieve the required security goals.

Proposition 1. *The proposed protocol ensures mutual authentication between the Device D and the Authentication Server AS in the authentication phase, and between D and the Access Point AP in the fast reconnection phase.*

Proof. In authentication phase, mutual authentication between Device D and Authentication Server AS is achieved when AS successfully verifies the received credentials message $\{Mid_i, M_1, HA_1\}$ afterwards verifying the $HA_1^* = HA_1$, $MSId_i^* = MSId_i$, $R_1^* = R_1$, and $I_u^* = I_u$ thereby authenticating D, and when D correspondingly verifies message $\{M_2, HA_2\}$ by confirming $HA_2^* = HA_2$ and $I_u^{N*} = I_u^N$ thereby AS is authenticated by device D. Thus, this protocol facilitates in mutual authentications. Similarly, in the fast reconnection phase, mutual authentication between D and the Access Point AP is established when AP validates $\langle \{MTId_i^T, EFC^1, HT_1\} \rangle$ by ensuring $HT_1^* = HT$, and $Mid_i^{New*} = Mid_i^{New}$, authenticating D, and when D verifies $\langle MTId_i^{NEW}, EFC^2, HT_2 \rangle$ by checking $MTId_i^{NEW*} = MTId_i^{NEW}$ and $R^{2*} = R^2$ thereby authenticating AP.

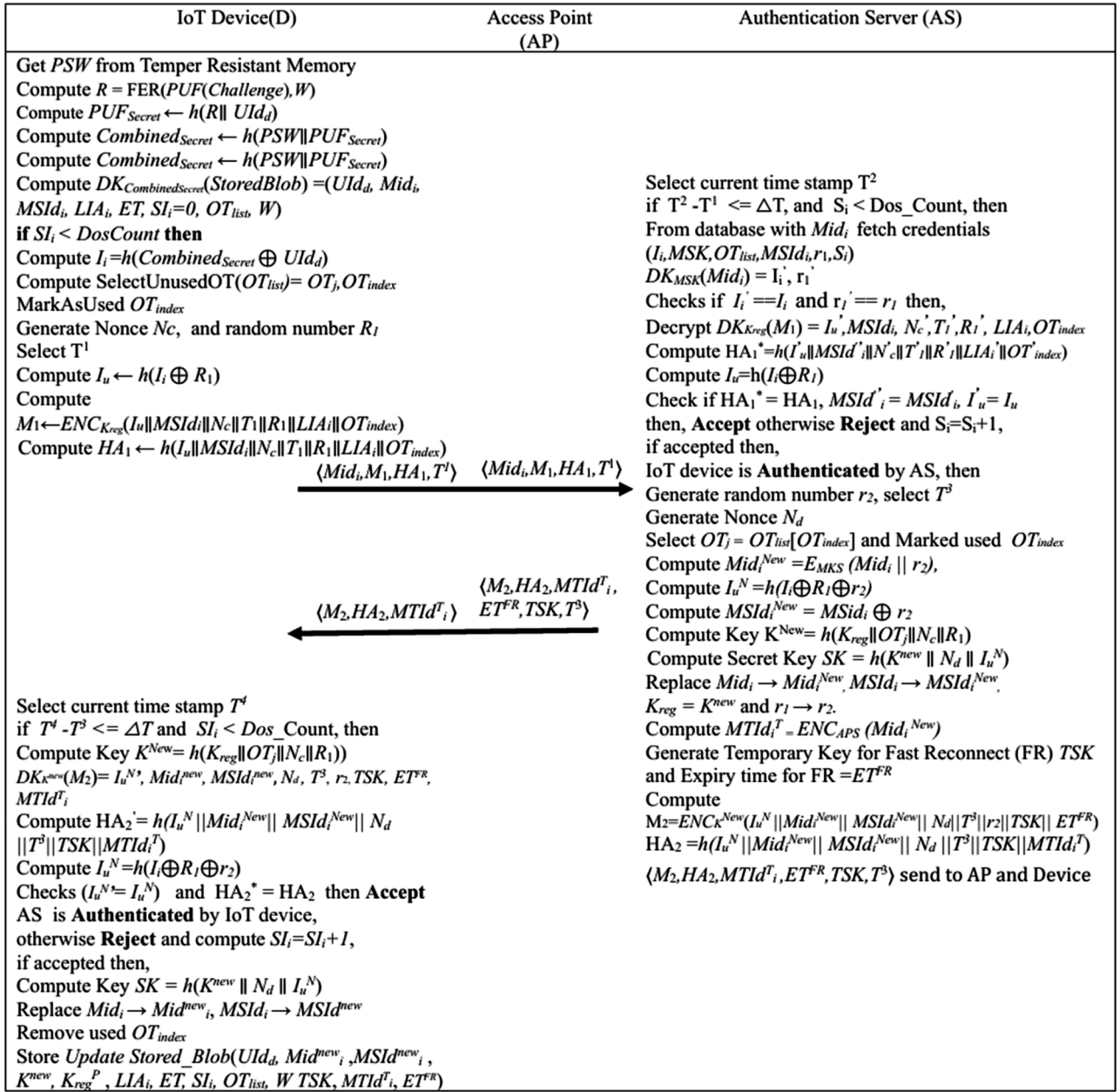


Fig. 6. Authentication phase.

Proposition 2. The proposed protocol ensures identity protection (anonymity) and resists traceability attacks.

Proof. During authentication, the device identity is concealed as $I_i = h(PSW || Uid_d)$, and further masked as $I_u = h(I_i \oplus R_1)$ while the server identity is protected as $MSId_i = E_{MSK}(Sid_i || r_1)$. The inclusion of fresh random nonces R_1 and r_1 in each session ensures that all identity-related parameters are randomized per protocol run. Consequently, an adversary intercepting protocol messages cannot recover the true identities of the device or server. Furthermore, because the masked identifiers change probabilistically in every session, an eavesdropper cannot correlate different sessions to trace the device or server over time. Thus, the protocol provides robust identity anonymity and is resilient to traceability attacks.

Proposition 3. Proposed protocols provide resistance from replay attacks.

Proof. The proposed protocol is resilient to replay attacks through the integration of freshness parameters in each exchanged message,

specifically timestamps and random nonces. Each message is validated by checking temporal freshness, such as verifying that $T^2 - T^1 = \Delta T$, falls within an acceptable window, and by confirming the correctness of challenge values such as $r_1' = r_1$. If any freshness check fails, the protocol immediately aborts the session. This mechanism ensures that previously captured messages cannot be successfully re-injected into a new session, thereby providing strong resistance against replay attacks.

Proposition 4. Proposed protocols provide resilience against man in the middle attacks.

Proof. The proposed protocol resists man-in-the-middle (MITM) attacks due to the cryptographic binding of session-specific secrets and integrity-protected message exchanges. An adversary attempting to forge a message such as altering $\{Mid_i, M_1, HA_1\}$ to $\{Mid_i^*, M_1^*, HA_1^*\}$ faces multiple barriers. First, the masked identity Mid_i changes each session and is controlled by the Authentication Server. Second, even if an adversary reuses a previously observed Mid_i , constructing a valid M_1 and its hash HA_1 requires knowledge of the protected secrets I_u and



Fig. 7. Fast reconnect phase.

$MSId_i$, where $I_u = h(I_i \oplus R_1)$ and $I_i = h(PSW || Uid_u)$. These values depend on both the device password PSW and a fresh random nonce R_1 , which the adversary cannot simultaneously guess in polynomial time. Moreover, all critical credentials are encrypted and accompanied by a cryptographic hash, making it infeasible for an attacker to generate a consistent forged message that would pass verification. Consequently, the protocol ensures that no adversary can successfully impersonate a legitimate party or tamper with messages undetected, providing strong resilience against MITM attacks.

Proposition 5. *Proposed protocols provide protection from ephemeral secret leakage (ESL).*

Proof. The proposed protocol provides protection against Ephemeral Secret Leakage (ESL) attacks. Even if an adversary compromises short-term session parameters such as random nonces R_1, r_1, r_2, N_b, N_c and timestamps T_1, T_2, T_3, T_4 the session key $SK = h(K^{new} || N_d || I_u^N)$ remains secure. This is because I_u^N is derived from the long-term device secret PSW and cannot be computed without it. The protocol ensures that short-term (ephemeral) secrets alone are insufficient to reconstruct the session key, and simultaneous exposure of both ephemeral and long-term secrets is assumed to be infeasible within the threat model. Thus, the protocol maintains session confidentiality even under ESL conditions.

Proposition 6. *Proposed protocols provide protection from Perfect Forward Secrecy.*

Proof. Proposed protocol provides perfect forward secrecy against

the compromise of long-term secrets stored on the device (e.g., PSW, OT_{list}), and the previous session keys remain secure. Forward Secrecy with respect to device compromise: Compromise of all secrets on the IoT device ($PSW, OT_{list}, K^{new} \leftarrow h(K_{reg} || OT_j || N_c || R_1), I_u^N \leftarrow h(I_i \oplus R_1 \oplus r_2)$ and secret keys) does not allow an attacker to compute past session keys (SK). The protocol's use of one-time pre-nonces (OT_{list}) provides a layer of protection. Even with SKS and MSK, the final session key $SK = h(K^{new} || N_d || I_u^N)$ on OT_j (from the one-time list) and fresh nonces (N_c, N_b, R_1) which are erased after the session. If the OT_{list} was securely deleted from the server after being sent to the device, and the device's OT_{list} is also compromised, then the chain is broken for sessions using already-consumed OT_j values. K^{New} is changing in each run of the authentication phase as $K^{New} \leftarrow h(K_{reg} || OT_j || N_c || R_1)$. It is hard to identify the current K^{New} due to the previous K^{New} .

If the attacker is able to get a long term secret like $\langle PSW, MSK, Uid_u \rangle$ but not able to compute the session key $SK = K^{New} \oplus N_d \oplus I_u^N$ due to I_u^N and K^{New} . K^{New} is changing in each run of the authentication phase as $K^{new} \leftarrow h(K_{reg} || OT_j || N_c || R_1)$. It is hard to identify the current K^{New} due to the previous K^{New} as K and Nonce N_c . Since the computation of session keys depends on both long-term and short-term secret keys, it is challenging to guess or obtain these credentials simultaneously. Consequently, the suggested protocol is safeguarded against assaults from Perfect Forward Secrecy.

Proposition 7. *Proposed protocols provide protection from stolen device attacks.*

Proof. The protocol offers robust defence against device theft by

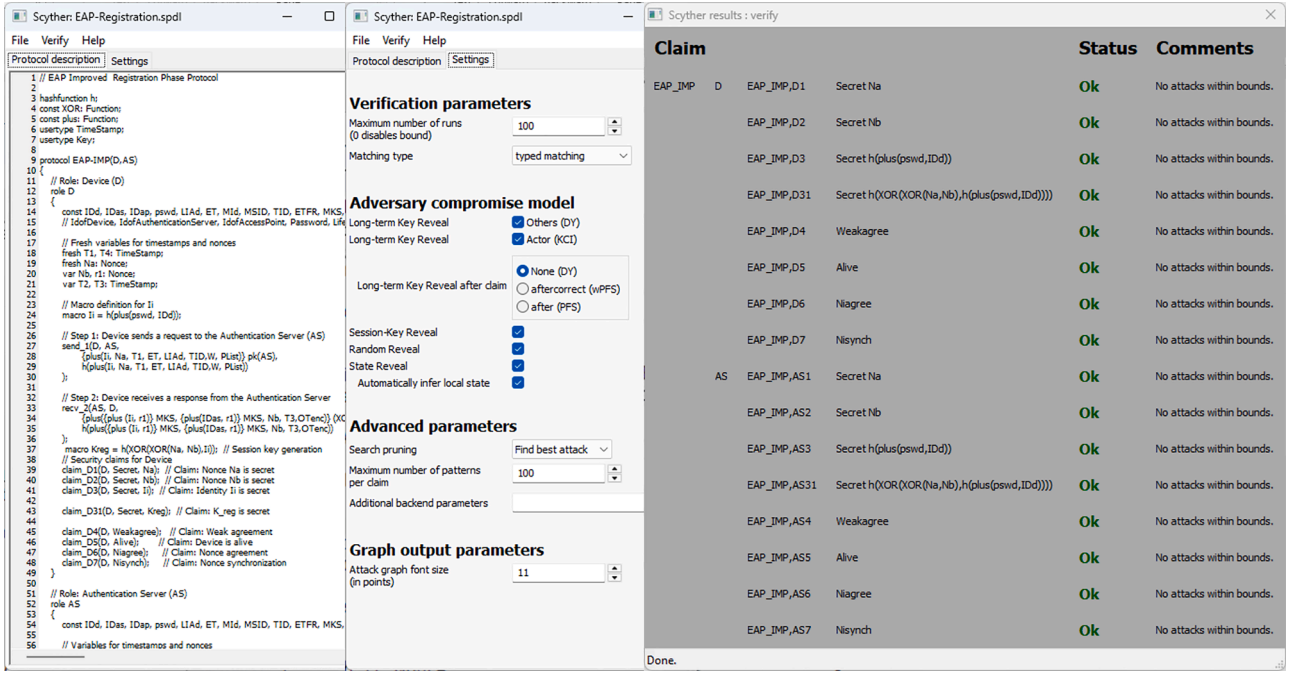


Fig. 8. Scyther result for registration protocol.

utilising hardware security mechanisms. The device utilizes a Physically Unclonable Function (PUF) and a tamper-resistant secure element to derive and protect its root secret, ensuring that critical credentials like the password PSW are never stored statically. Stored credentials are encrypted with $Combined_{Secret} = h(PSW || PUF_{Secret})$. Physical treft yields only encrypted data, requiring both password and PUF measurement from untampered hardware. Thus, the proposed protocol is protected from stolen device attacks.

Proposition 8. *Proposed protocols provide protection from desynchronization attack.*

Proof. An adversary may attempt to desynchronize the device DD and Authentication Server ASAS by intercepting or modifying messages $msg^1 = \{Mid_i, M_1, HA_1, T^1\}$ or $msg^2 = \{M_2, HA_2, T^3\}$ during authentication. The protocol mitigates such attacks as follows:

1. msg^1 is interrupted, and that message not received AS either never receives it or detects invalidity upon verification (e.g., incorrect hash HA_1 or timestamp). AS then aborts the session **without updating its current key**. After a timeout period $2\Delta T$, D re-initiates authentication using the same key K , allowing AS to recognize and process the new request seamlessly.
2. If msg^2 is blocked, D times out and restarts authentication. Although AS may have already updated to a new key K^{new} , it retains the previous key for a limited window. Upon receiving the retransmitted msg^1 , AS first attempts decryption with K^{new} , if this fails, it tries the previous key. Successful decryption indicates a prior session failure, prompting AS to resend msg^2 and re-establish synchronization.
3. If msg^2 is modified in a man-in-the-middle attack, D will detect the invalid hash HA_2 or timestamp and abort. AS, having already updated to K^{new} , retains previous key. When D retries, AS can again fall back to previous key to recognize the legitimate retry, recover the session, and re-transmit msg^2 correctly.

In all cases, the protocol ensures that at least one party retains the ability to recognize a legitimate retry, preventing permanent desynchronization. The combination of key versioning, retry fallback logic, and integrity verification provides robust protection against desynchronization attacks.

Proposition 9. *The proposed protocol incorporates mechanisms to mitigate Denial-of-Service (DoS) attacks through stateful attempt counting and throttling policies.*

Proof. The Proposed protocol establishes DoS counters on both the Authentication Server (S_i) and the IoT Device (SI_i). These counters increase with each aborted or unsuccessful authentication attempt. An adjustable threshold, DoS_Count , delineates the maximum allowable failures within a specified time frame. Upon reaching this threshold, the relevant entity (server or device) implements a lockout period, temporarily halting additional authentication attempts from the offending source. This inhibits an adversary from depleting computational resources by frequent, invalid authentication attempts. The threshold and lockout duration are dictated by policy and can be synchronised with standards like NIST SP 800-63B, Grassi et al. [33] which advise a maximum of five consecutive failures for high-risk systems and 10 for lower-risk situations. The protocol mitigates resource depletion attacks by restricting the frequency of cryptographic operations (e.g., decryption and verification) per client, hence preserving availability for genuine users.

Proposition 10. (Privileged Insider Attack Resilience). *The proposed protocol mitigates privileged insider attacks.*

Proof. The proposed protocol ensuring that no plaintext or reversible long-term device secrets are stored on the Authentication Server (AS). Instead, the AS retains only cryptographically transformed values (e.g., hashed passwords or encrypted one-time nonce lists), while session keys are derived using ephemeral, non-persistent random values exchanged during each authentication run. Consequently, even an insider with full database access cannot extract usable device credentials, impersonate a legitimate device, or compute active or past session keys, as the necessary cryptographic material is either protected by one-way functions or exists only ephemerally on the device side, thereby neutralizing the threat of credential misuse from within the trusted infrastructure.

6. Formal analysis through Scyther tools

Scyther is a formal verification tool designed for the security analysis of cryptographic protocols, capable of identifying logical flaws through

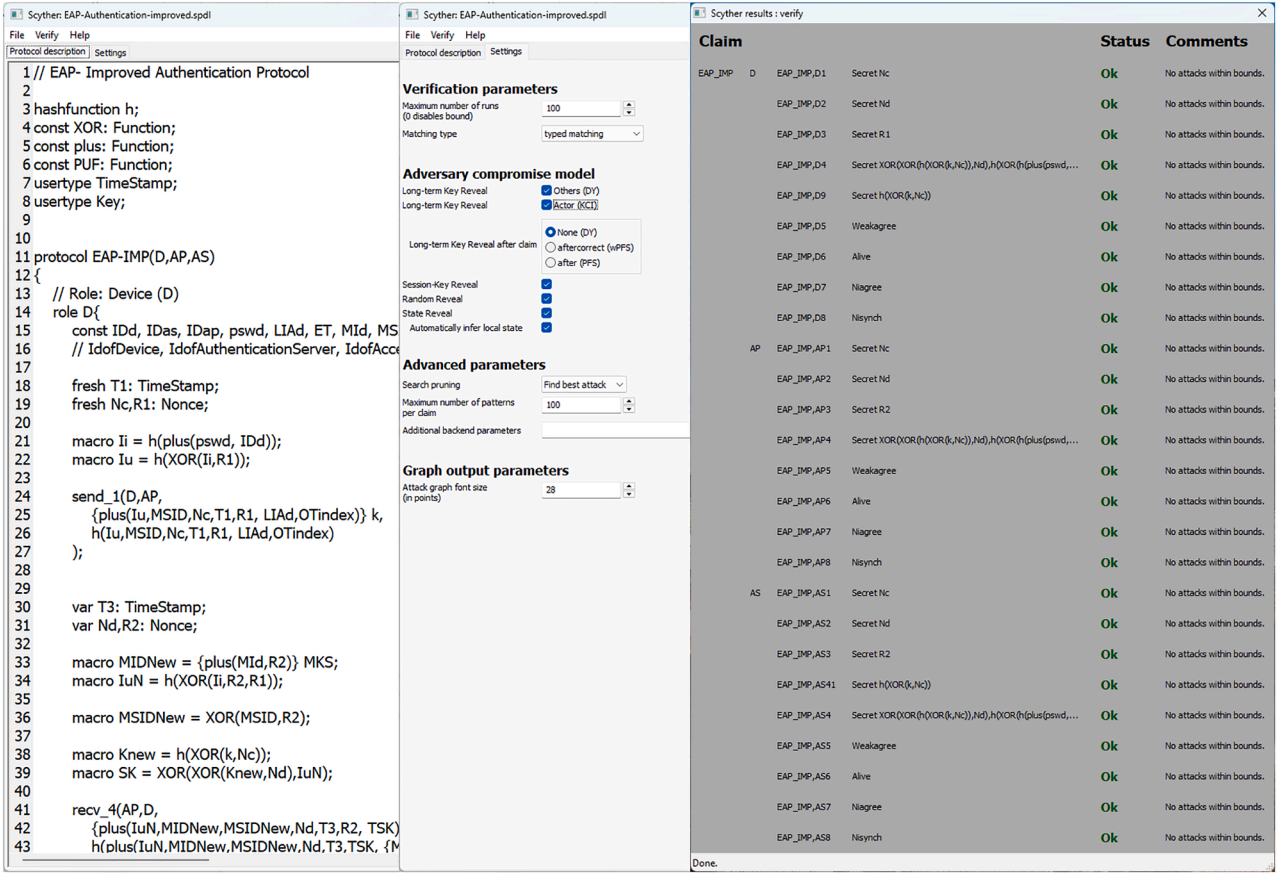


Fig. 9. Scyther result for authentication protocol.

model checking under the Dolev-Yao adversary model. It verifies fundamental security properties, including confidentiality, authentication, and synchronisation, over unbounded protocol sessions a capability that enables the generalisation of attack patterns beyond finite-state analysis and enhances scalability. In addition to automated verification, Scyther can generate graphical attack traces (e.g., via SPICY) that visually illustrate vulnerabilities, thereby aiding protocol debugging. It assumes flawless cryptography and hence concentrates exclusively on design-level vulnerabilities rather than implementation deficiencies. The integration of automation, accessibility, and comprehensive output renders Scyther an indispensable instrument for pre-deployment certification of security methods.

Scyther was built with advanced search pruning for protocol verification to effectively find optimal attack tactics, restricting the investigation to a maximum of 100 patterns per security claim. Verification was performed within the Dolev-Yao threat model, augmented by supplementary adversary compromise capabilities, with a limit of 100 iterations per scenario. As shown in Figs. 8 and 9, the Scyther outputs for both the registration and authentication phases confirm that no attacks were discovered within the specified bounds. The findings indicate that the protocol maintains its security attributes, including absolute forward secrecy, even in scenarios including session key disclosure.

7. Formal analysis through ROR oracle model

The Real-or-Random (ROR) model is a rigorous formal framework employed to evaluate the semantic security of authentication mechanisms and key exchange protocols. In this model, an adversary attempts to distinguish between real protocol executions (using genuine

cryptographic keys) and simulated executions (using random values). The adversary interacts with protocol instances through oracle queries such as Execute (eavesdropping), Send (active injection), and Test (challenge). In the Test query, the adversary receives either the actual session key or a random string of equal length. The protocol is considered secure if the adversary's advantage in distinguishing between the two is negligible.

It emphasizes on indistinguishability, ensuring that secrets remain private even if other parameters are compromised. The proposed protocol relies on this model to demonstrate its resistance to eavesdropping, forgeries, and hidden leakage. In contrast to previous approaches (such as IND-CPA), ROR is intended to explicitly handle session security in dynamic environments.

The proposed protocol's security is attacked by an adversary (A) on a real-time system; 'A' attempts to retrieve or produce the communicating session key. Adversary 'A' uses a variety of methods to compromise the security of the proposed authentication and key exchange protocol 'P'. However, the attacker can only access information that is publicly available or steal IoT devices that are no longer operational. Adversary A creates an oracle system to conduct these experiments.

We consider a probabilistic polynomial-time adversary 'A' who can interact with protocol instances through various oracles. The protocol involves three entities: IoT device (D), Access Point (AP), and Authentication Server (AS).

Let the three participants Y_I^x , Y_{AP}^y and Y_{AS}^z denotes the xth, yth and zth instances of IoT device (I), Access Point (AP) and Authentication sever (AS) respectively. All three Y_I^x , Y_{AP}^y and Y_{AS}^z oracle shares the communication session key $SK \leftarrow h(K^{new} \parallel N_d \parallel I_{II}^N)$. The oracle Y_I^x ,

Y_{AP}^x and Y_{AS}^x remains fresh whenever the session key not exposed with the adversary through any oracle query. To crack the security of session keys and authentication messages, the Adversary 'A' can execute simple queries on the simulator [27]. The simulator enumerates the following queries:

Execute (Y_I^x, Y_{AP}^y): Adversary 'A' executes passive eavesdropping attacks where adversary intercept and disclose all of messages between AP and AS.

Send (Y_I^x, Y_{AP}^y): This active attack allows 'A' to intercept and receive a reply message directly from participant Y_I^x . Y_{AP}^y then responds to 'A' in accordance with the rules of the protocol.

Reveal (Y_I^x): The Adversary 'A' executes the Simulation query to get the Y_I^x generated current secret key.

Corrupt (Y_I^x): This query an Adversary can learn the stored secret in IoT devices. The adversary

who is capable of corrupting one or several authentication credentials of IoT devices. This corruption simulation query simulates 'A's' capacity to access IoT device secret information that has been stored on a IoT devices.

if $a = 1$, query returns password PSW of IoT devices
if $a = 2$, query return secret parameters stored in IoT device.
If $a = 3$ query return the PUF response with challenges.

Test (Y_I^x, Y_{AP}^y): This query assesses the semantic security of a secret key shared between a IoT device and a Authentication server, specifically focusing on the indistinguishability in the Real-or-Random (ROR) model. Initially, 'A' conducts an unbiased coin toss 'b', and the outcome of this determines the output of the test question, which is only known to 'A'. In this case if $b = 1$, the 'A' can obtain the secret key and if $b = 0$, the adversary obtains random strings of same length.

Definition 1. (Semantic Security) Our authenticated protocol's semantic security is defined as negligible advantages achieved for acquiring or extracting the secure secret key by the adversary.

Success(A) is defined as the occurrence of an event where an adversary 'A' successfully executes a Test (A probabilistic polynomial time adversary that performs oracle queries and interacts with its instance of running participants Y_I^x, Y_{AP}^y). In this event, the adversary 'A' guesses the bit b' , if $b' = b$, then Adversary 'A' has broken the semantic security of the authentication and key establishment protocol (AKE) [27].

The advantage function of adversary 'A' in breaking the semantic security of our protocol p by guessing bit b' is defined as follows:

$$Adv_p^{AKE}(A) = |\Pr[Success(A) - 1] = |2\Pr[b = b'] - 1||$$

Theorem 1. Let A be adversary running in polynomial time t against our protocol P to break the semantic security under ROR model can be broken by A with the Advantages function

$$Adv_p^{AKE}(A) \leq \frac{q_h^2}{2^h} + \frac{(q_s + q_e)^2}{2^r} + \frac{q_{puf}^2}{|PUF|} + 2 \cdot \frac{(q_s + q_d)^2}{|D|} + 2Adv_p^{ECDLP}(n)$$

Where q_s =numbers of send queries,

q_h , =numbers of hash queries,
 q_e =numbers of execute queries,
 q_d =numbers of dictionary oracles queries,
 q_{puf} = number of PUF queries,
 $|D|$ = size of password dictionary,

$|PUF|$ = space of PUF responses,
 l_h = length of string of hash
 l_r = length of string of random nonce and

$Adv_p^{ECDLP}(n)$ denotes the advantages of adversary to breaking of elliptic curve discrete logarithmic problems.

Proof: In this proof, we define a sequence of games G_i where $i = [0$ to $5]$. Let S_i denotes the event wherein the bit b in game G_i is able to be guessed by the adversary, we have

Game G_0 : This game represents the real attack in the random oracle model. The bit b is chosen randomly at the beginning. we have,

$$Adv_p^{AKE}(A) = |2\Pr[S_0] - 1| \quad (1)$$

Game G_1 : This game as an eavesdropping attack, where adversary A intercepts communication between the IoT devices and the access point to compute $SK \leftarrow h(K^{new} \parallel N_d \parallel I_u^N)$ without knowing the K^{new} , N_d, I_u^N . Since these values remain secret, hence, we have

$$\Pr[S_1] = \Pr[S_0] \quad (2)$$

Game G_2 : we add the simulation of send & the hash oracles based on G_1 . The games Game G_1 and G_2 are identical except for the hash and send queries. Discovering the collision in secret symmetric key cryptography is $\frac{q_h^2}{2^{1+l_h}}$. The probability to detect the collision of random nonce is defined as $\frac{(q_s + q_e)^2}{2^{1+l_r}}$.

According to birthday paradox, we have

$$|\Pr[S_1] - \Pr[S_2]| = \frac{q_h^2}{2^{1+l_h}} + \frac{(q_s + q_e)^2}{2^{1+l_r}} \quad (3)$$

Game G_3 : Adversary 'A' execute an attack by simulating the process of querying the corrupt IoT devices to obtaining the stored password PSW through online dictionary attack.

$$\text{Sowehave} |\Pr[S_2] - \Pr[S_3]| = \frac{(q_s + q_d)^2}{|D|} \quad (4)$$

Game G_4 (PUF Security): The adversary want acquire the control over IoT device to want to get the PUF responses to match the security credentials. The advantage of 'A' to get the correct PUF responses as

$$|\Pr[S_3] - \Pr[S_4]| \leq \frac{q_{puf}^2}{|PUF|}$$

Game G_5 Simulates attacks on the registration phase encryption. Breaking this requires solving the ECDLP:

$$|\Pr[S_4] - \Pr[S_5]| \leq Adv_p^{ECDLP}(n)$$

All the oracles have been simulated within the game. Adversary A successfully determines the value of b by querying the test oracle. Only adversary A remains uncertain about the guessing the correct value of bit b , resulting in the following consequences.

Table 2

Comparison of Security feature and functionality analysis.

	[9]	[19]	[5]	[34]	[12]	[8]	[30]	[25]	Ours
Mutual authentication	✓	✓	✓	✓	✓	✓	✓	✓	✓
Perfect forward secrecy	✓	✓	✓	✓	X	✓	✓	✓	✓
Identity protection	✓	✓	✓	X	✓	✓	✓	✓	✓
Tractability	✓	✓	X	X	X	X	✓	✓	✓
Ephemeral secret leakage	✓	✓	X	X	X	✓	X	✓	✓
Privileged insider attack	✓	✓	✓	✓	X	X	X	X	✓
Desynchronization attack	✓	✓	✓	X	✓	✓	✓	X	✓
Replay attack	✓	✓	✓	✓	✓	✓	✓	X	✓
DOS attack	X	✓	X	X	X	X	X	X	✓
Stolen device attack	X	✓	X	✓	X	X	X	X	✓
Fast reconnect	✓	✓	X	✓	✓	X	✓	X	✓
Insecure channel for registration	✓	X	X	X	✓	X	✓	X	✓

✓: Secure against attack/available. X: Insecure against attack/Not available.

$$\Pr[S_6] = \frac{1}{2}$$

Now, $|\Pr[S_0] - \frac{1}{2}| = |\Pr[S_0] - \Pr[S_6]| = |\Pr[S_0] - \Pr[S_1] + \Pr[S_1] - \Pr[S_2] + \Pr[S_2] - \Pr[S_3] + \Pr[S_3] - \Pr[S_4] + \Pr[S_4] - \Pr[S_5] + \Pr[S_5] - \Pr[S_6]|$
 By the Triangular inequality

$$\begin{aligned} \left| \Pr[S_0] - \frac{1}{2} \right| &= |\Pr[S_0] - \Pr[S_1] + \Pr[S_1] - \Pr[S_2] + \Pr[S_2] - \Pr[S_3] + \Pr[S_3] - \Pr[S_4] + \Pr[S_4] - \Pr[S_5] + \Pr[S_5] - \Pr[S_6]| \\ &= \sum_{i=0}^{i=6} |\Pr[S_i] - \Pr[S_{i+1}]| \end{aligned} \quad (5)$$

From Eq. (1), (3) we have

$$\begin{aligned} Adv_p^{AKE}(A) &= |2\Pr[S_0] - 1| \\ \frac{1}{2} Adv_p^{AKE}(A) &= \left| \Pr[S_0] - \frac{1}{2} \right| \\ &\leq \sum_{i=0}^{i=6} |\Pr[S_i] - \Pr[S_{i+1}]| \\ &\leq \frac{q_h^2}{2^l h} + \frac{(q_s + q_e)^2}{2^l r} + \frac{q_{puf}^2}{|PUF|} + 2 \cdot \frac{(q_s + q_d)^2}{|D|} + 2 Adv_p^{ECDLP}(n) \\ \text{So, } Adv_p^{AKE}(A) &\leq \frac{q_h^2}{2^l h} + \frac{(q_s + q_e)^2}{2^l r} + \frac{q_{puf}^2}{|PUF|} + 2 \cdot \frac{(q_s + q_d)^2}{|D|} + 2 Adv_p^{ECDLP}(n) \end{aligned} \quad (6)$$

8. Performance analysis

This section analyses the comparison of security features, computational costs, and communication costs with state-of-the-art protocols. Additionally, evaluate the energy usage of different IoT devices employing distinct protocols.

8.1. Comparison of security feature

The comparison analysis reveals that the proposed protocol offers a significant security benefit over existing protocols. This protocol uniquely fulfils all twelve assessed security criteria, encompassing mutual authentication, perfect forward secrecy, identity protection, and resilience against traceability, ephemeral secret leakage, insider threats, desynchronisation, replay, DoS, and stolen device attacks, while additionally facilitating rapid reconnection and registration via an insecure channel. Conversely, each of the cited protocols ([5,8,9,12,19,25,30,34]) is deficient in at least one category, with numerous protocols lacking essential features such as DoS mitigation, resistance against stolen devices, or secure registration across insecure channels. This

establishes the proposed protocol as a distinctly comprehensive and pragmatic solution for safe authentication in IoT-WLAN systems.

A comparison of security and functionality with existing protocols is presented in Table 2. Symmetric-key authentication systems [5,8], and [25] typically provide less security assurances than asymmetric-key protocols [9,12], and [30]. Although the majority of current tech-

niques are resistant to MITM and replay attacks excluding [25] considerable flaws persist. For example, protocol [12] lacks perfect forward secrecy, and all but [9] lack the essential defences against Denial-of-Service (DoS) and stolen device attacks. Furthermore,

Table 3

Comparison of cryptographic computational cost for protocols.

Protocol	Device side	Server side	Total Computation cost (ms)
[9] Yadav et al. (2022)	6E	8E+H	14E+H ≈ 0.7901
[19] Yadav et al. (2024)	2E+3H+PE	2E+6H+PE	4E+9H+2PE ≈ 17.908
[5] Shojaie et al. (2017)	2E+H+PE	E+H+PE	3E+2H+2PE ≈ 13.088
[34] Wang et al. (2025)	2PE+4H+2V+2M	2PE+4H+2V+2M	4PE+8H+4V+4M ≈ 30.2692
[12] Amit et al. (2016)	4E+H+V	4E+2H+V	8E+3H+2V ≈ 0.9218
[8] Pawan et al. (2020)	PE+E+DH	PE+DH	E+2PE+2DH ≈ 12.839
[30] Tsobdjou et al. (2021)	5H+3PM	3H+3PM	8H+6PM ≈ 6.2029
[25] Baruah et al. (2020)	PM+5H	2PM+6H	3PM+6H ≈ 2.6148
Ours	4E+8H+PF	6E+5H	7E+11H ≈ 0.6774

Table 3 symbols used for calculation of computational cost of H: hash Function, E: AES encryption/decryption, PE: PKI encryption/decryption, DH: Diffie-Hellman, PM: Point Multiplication, V: Verification cost, PF: PUF = 0.7*H [35].

Table 4

Computational cost of cryptographic operations on devices.

Computational Specification	Raspberry Pi 4 (ms)	Raspberry Pi B/B+ (ms)	Raspberry Pi Zero W	Intel i5-1135 (ms)
h(SHA256)	0.011	0.013	0.216	0.0053
mac(HMAC-SHA-256)	1.423	1.572	1.895	0.065
e(AES-CBC-256 Enc)	0.0653	0.0752	0.0956	0.032
e(AES-CBC-256 Decryption)	0.097	0.132	0.237	0.0673
Point Multiplication	1.56	1.62	3.69	0.484
Diffie-Hellman	3.052	3.77	6.714	1.002
RSA Enc	8.22	9.78	17.24	4.64
Verification Cost	0.252	0.30744	0.49896	0.125

ms: millisecond.

common shortcomings across the surveyed protocols include inadequate resistance to traceability, ephemeral secret leakage, and privileged insider attacks. While protocols [9,12], and [30] facilitate registration via an insecure channel, this design leads to vulnerabilities to many known attacks. Furthermore, the critical fast reconnection function is not supported by references [5,8], and [25]. These collective constraints emphasise the necessity for a more robust and comprehensive authentication system.

8.2. Comparison of cryptographic computational cost

Let the cryptographic computational cost of hash function, AES encryption/decryption, public key based like RSA/ECC encryption/decryption, Diffie-Hellman, Point Multiplication, Verification cost be the H, E, PE, DH, PM, and V respectively. Then we compare for cryptographic computational cost of exiting schemes with our proposed protocol which is listed in Table 3[27]. For the calculation of total time as a testbed, the IoT device is taken as the Raspberry Pi 4 model of 2GB RAM and at the authentication server-side as the Intel i5-1135G7 processor with frequency of 2.4 GHz of 8GB of RAM then the total computation time in millisecond shown in Table 3. Total computation time shows that the existing schemes like [23,25,30] have a lower computation cost compared with the proposed protocol, but these schemes is vulnerable to various attacks. Our proposed protocol provides a lower cryptographic computational cost than existing authentication protocols. Details of computational time at different IoT devices and authentication server are shown in Tables 4 and 5[27]. The access point does not perform any computation in the authentication phase other than just forwarding the message. Access points at the fast reconnect phase do computation which is not included in the authentication phase.

8.3. Experimental setup

We evaluated the computational performance of the proposed authentication protocol using various hardware platforms that represent standard IoT devices and server architecture. The hardware of the IoT gadget comprised:

- 1) Raspberry Pi 3B+/B: Features a quad-core 64-bit Broadcom BCM2837 ARM Cortex-A53 system-on-chip operating at 1.2 GHz with 1 GB of RAM.
- 2) Raspberry Pi 4: Equipped with a Broadcom BCM2711 quad-core Cortex-A72 (ARM v8) 64-bit SoC operating at 1.8 GHz and 2 GB of RAM.
- 3) Raspberry Pi Zero W: Features a single-core ARM Cortex-A53 CPU operating at 1.1 GHz and 512 MB of RAM.

The Authentication Server was simulated on a desktop computer equipped with an Intel i5-1135G7 processor (2.4 GHz base clock) and 8 GB of RAM. Table 4 displays the recorded execution times (in milliseconds) for different cryptographic primitives across each platform. The Raspberry Pi 3B+/B demonstrated marginally greater overhead than the Pi 4, however the resource-limited Pi Zero W necessitated execution durations 2–3 times longer than those of the Pi 4.

The experiment attempted to evaluate the cryptographic computational cost of the proposed protocol in comparison to existing techniques on the specified hardware. Cryptographic operations, such as AES-256-CBC encryption and decryption, RSA and ECC encryption and decryption, elliptic-curve point multiplication, and signature verification, were executed utilising Python 3 cryptographic libraries. To guarantee statistical reliability, each action was performed 1000 times utilising the timeit module, which executes the code iteratively and provides the average execution duration. This methodology offers a consistent and precise assessment of the computational burden imposed by each cryptographic primitive during the authentication procedure.

As shown in Table 5, the existing authentication schemes predominantly employ an AES key size of 128 bits, which offers a lower security margin compared to larger key sizes. For identity, nonce, and random number parameters, our protocol adopts a 128-bit size, whereas most comparable schemes utilize only 32 bits, except for [19] and [8], which use 160 bits. Notably, protocol [9] relies on the SHA-1 hash function (160-bit output), which is widely recognized as cryptographically weakened and susceptible to collision attacks. Password lengths across the schemes are generally standardized at 128 bits. Regarding asymmetric cryptography, protocols employing ECC typically use 160-bit keys, while RSA-based schemes predominantly utilize 1024-bit keys, except for [8,19], and [23], which implement the more secure 2048-bit RSA key size. This parameter comparison highlights notable variations in cryptographic strength among existing approaches.

Table 5 describes the required time for running protocols on different hardware. The proposed protocol takes less time than existing authentication protocols except [23,30], and [25], which are less secure and vulnerable with many known attacks and fewer features. Also, the key size of the cryptographic operation and the length of the password are higher, which provide better security than existing schemes that have a lesser key size. Fig. 8 shows the computational cost at the device side and authentication server side. Fig. 10 shows the testbed situation where if the Raspberry Pi 4 acts as an IoT device and the Intel i5-1135 acts as an authentication server, the existing schemes have the computational time at the device level and the authentication server level and combine the total time taken. In this case our proposed protocol takes as time for $2E+2D+8H+PF \approx 2 \times 0.0653+2 \times 0.097+8 \times 0.011+0.7 \times 0.011=0.4203$ ms, at IoT device side and at server side $3E+2D+5H \approx 3 \times 0.032+2 \times$

Table 5

Comparison of computational cost for existing protocols.

Protocol	[9]	[19]	[5]	[34]	[12]	[8]	[30]	[25]	Ours
Raspberry Pi 4	0.4869	8.496	8.3933	20.338	0.5876	11.35315	4.735	1.615	0.4203
Raspberry Pi B/B+	0.6216	10.13	10	23.9857	0.73484	13.6536	4.925	1.685	0.5275
Raspberry Pi Zero W	0.9978	18.39	17.789	41.1298	1.38016	24.1203	12.15	4.77	2.5444
Intel i5-1135	0.3032	9.406	4.695	9.9312	0.3342	1.486	1.4679	0.9998	0.2571
Total time (ms)	0.7901	17.9	13.088	30.2692	0.9218	12.83915	6.2029	2.6148	0.6774

ms: millisecond.

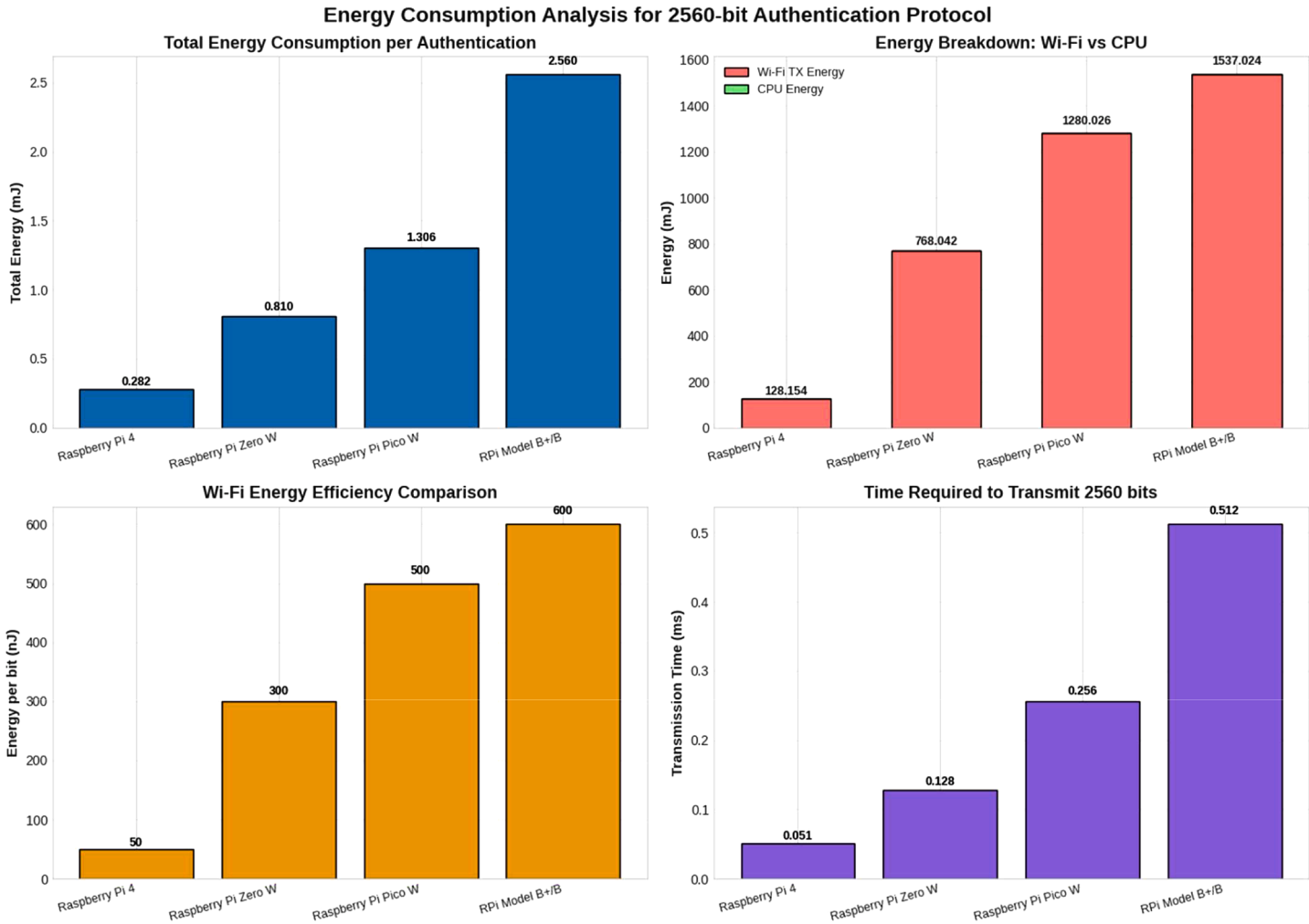


Fig. 10. Energy consumption by radio WiFi transmission by smart devices.

Table 6

Symbol and key size.

Symbol	Symbol size		bits
	bits	Symbol	
Identity	128	Nonce (N_i)	128
Timestamp	32	Hash Function	256
Expiry Time	32	Password	128
AES key	256	ECC key	160

$0.0073 + 5 \times 0.0053 = 0.2571$ ms, and the total time is $(0.4203 + 0.2571) 0.6774$ ms [35].

8.4. Communication cost

Table 6 details the cryptographic symbol sizes and key lengths employed by the protocols, while Table 7 and Fig. 9 present a comparative analysis of message communication in bits. In the proposed protocol, the message exchange between the IoT device and the access point comprises the tuples $\langle M_{id}, M_1, HA_1, T^1 \rangle$ and $\langle M_2, HA_2, T^3 \rangle$ amounting to **2560 bits**. This communication cost is marginally higher than that of protocols with comparable security levels, such as [9,30], and [25]. The increased overhead is justified by the protocol's provision of enhanced security features not uniformly supported by existing schemes, including perfect forward security, traceability, protection from ephemeral secret leakage, protection from privileged insider attacks, and fast reconnect. Many existing protocols achieve lower bit counts by using shorter, 32-bit identities and nonces, which introduces security vulnerabilities. Furthermore, protocol [9] utilizes a 160-bit hash function (likely SHA-1), which is now considered cryptographically weak.

The proposed Protocol generates approximately 800 more bits of transmission overhead each authentication session relative to existing schemes. For a device like the Raspberry Pi 4, which has a Wi-Fi radio that uses around 50 nJ each transmitted bit, this results in an additional energy expenditure of merely 0.04 Millijoules (mJ) per authentication. This minimal energy expenditure signifies a beneficial trade-off, offering considerably enhanced security assurances with a negligible effect on gadget power consumption and operational lifespan.

8.5. Energy consumption by IOT devices

Tables 8 and 9 are showing the computational power consumption and Wi-Fi radiations of IoT device like board Raspberry Pi W Zero, Raspberry Pi Pico W, Raspberry Pi B/B+ and Raspberry Pi 4 (RPI 4) [27]. Table 10 shows that the supported Wi-Fi Standard technology. As newer technology introduces the energy for transmission per bit reduces. RPi 4 supports the 802.11ac which have better data rate of 50 Mbps with less transmission cost of 50 nJ/bit compared to other supporting Wi-Fi standard like 802.11n and 802.11g. RPi 1 Model B+/B requires external Wi-Fi module which enhance the transmission power as compare with Raspberry Pi W Zero. It shows that better Wi-Fi technology reduces the transmission cost per bit.

Table 10 shows existing schemes consume the energy (mj=milli-joules) for transmission of message with three different IoT devices. Proposed protocol transmits the 2560 bit. As per Table 5, RPi 4 requires 0.4203 millisecond for cryptographic operation then energy consumed be $(2560 \times 0.00005 \text{ mj} + 3 \times 0.4203 \text{ mj}) 1.3889 \text{ mj}$. Similarly, Raspberry Pi W Zero requires for computation is 2.5444 ms and transmit 2560 bit then energy consumed $2560 \times 0.0003 \text{ mj} + 2.5444 \times 0.33 \text{ mj} = 1.6076 \text{ mj}$.

Fig. 11 provides a detailed computational performance analysis of

authentication methods across three IoT platforms: Raspberry Pi 4, Raspberry Pi Zero W, and Raspberry Pi 3 B/B+, evaluating eight existing protocols [5,8,9,12,19,25,30,34] in comparison to the suggested approach. The suggested protocol exhibits enhanced efficiency, with computation times of 1.39 ms (RPi 4), 1.61 ms (Pi Zero W), and 1.06 ms (RPi 3 B/B+), indicating performance enhancements ranging from 79.9 % to 93.0 % compared to the average of current protocols. Conversely, protocol [34] demonstrates the most computational overhead at 61.01 ms on the RPi 4—approximately 44 times slower than our solution—whereas protocol [8] necessitates 34.22 ms on the identical hardware. The layered analysis indicates a total computing overhead of merely 4.05 ms for the proposed protocol, compared to 150.15 ms for protocol [34] and 71.77 ms for protocol [8] across all devices. The performance ratio analysis indicates that current protocols are between $1.5 \times$ and $44 \times$ slower, with the RPi 3 B/B+ exhibiting the greatest significant disparity at $45.4 \times$ slower for protocol [34]. A heatmap visualisation validates the proposed protocol's uniform efficiency across hardware variants, indicated by the lightest colour bands, while simultaneously emphasising the specific vulnerability of resource-constrained devices such as the Raspberry Pi Zero W to inefficient designs. The proposed protocol ranks first in computational efficiency on both RPi 4 and RPi 3 B/B+, and fifth on Pi Zero W, yet maintains highly consistent execution times (range: 0.55 ms) compared to the significant variability of existing protocols (e.g., protocol [34] varies by 19.85 ms). The results confirm that the proposed protocol strikes a vital balance between computational efficiency and complete security, rendering it especially appropriate for energy-constrained IoT implementations where performance and robust authentication are important.

9. Limitations and future work

The proposed protocol has several limitations that inform future research directions:

1. Hardware Dependency: Stolen device protection assumes PUF and tamper-resistant hardware, which may be cost-prohibitive for ultra-low-cost nodes. Future work will explore software-based alternatives.
2. Quantum Threat: While symmetric primitives (AES-256, SHA-256) are post-quantum resilient, certain asymmetric components remain vulnerable. Integration of post-quantum cryptographic (PQC) primitives will be investigated.
3. Analysis Scope: Formal verification under the Dolev-Yao model does not cover implementation-level attacks (side-channel, fault injection). Physical-layer security analysis is needed.
4. Trust Assumptions: Security relies on trusted manufacturing and enrolment. Decentralized or verifiable enrolment protocols could reduce this single point of trust.

Table 8

Smart device CPU and Wi-Fi transmission power consumption.

Device	CPU Power (W)	Wi-Fi Power (W)	Wi-Fi Efficiency (nJ/bit)
Raspberry Pi W Zero	0.33	0.66	~300
Raspberry Pi Pico W	0.1	0.5	~500
RPi Model B+/B	2	1.00 (USB Dongle)	~400
Raspberry Pi 4	3	1	~50

Table 7

Comparison of communicational cost with state of arts protocol.

Protocol	[9]	[19]	[5]	[34]	[8]	[30]	[25]	Ours
Cost bits	1664	2784	2634	1312	3264	1296	1024	2560

Table 9

WiFi Transmission energy consumption.

Device	Wi-Fi Standard	TX Power	Data Rate	Energy/bit	2560-bit TX	CPU Energy (J)	TX Time for 2560 bits (ms)
RPi 4	802.11ac	1.0W	50 Mbps	~50 nJ/bit	0.1280 mJ	0.000154	0.051
RPi Zero W	802.11n	0.8W	20 Mbps	~300 nJ/bit	0.7680 mJ	0.000042	0.128
RPi Pico W	802.11n	0.5W	10 Mbps	~500 nJ/bit	1.2800 mJ	0.000026	0.256
RPi 1B/B+	802.g11g	1.2W	5 Mps	~800 nJ/bit	1.5360 mJ	0.001024	0.512

Table 10

Energy consumption(mJ) by IoT devices with state of art protocol.

Protocol	[9]	[19]	[5]	[34]	[12]	[8]	[30]	[25]	Ours
RPi 4	1.5439	25.628	25.311	61.0146	1.8092	34.222	14.269	4.8962	1.3889
RPi Zero W	0.8284	6.9028	6.6604	41.1632	0.7338	8.9389	4.3983	1.8813	1.6076
RPi 3 B/B+	1.9088	21.373	21.054	47.9717	1.8408	28.612	10.368	3.7796	1.0570

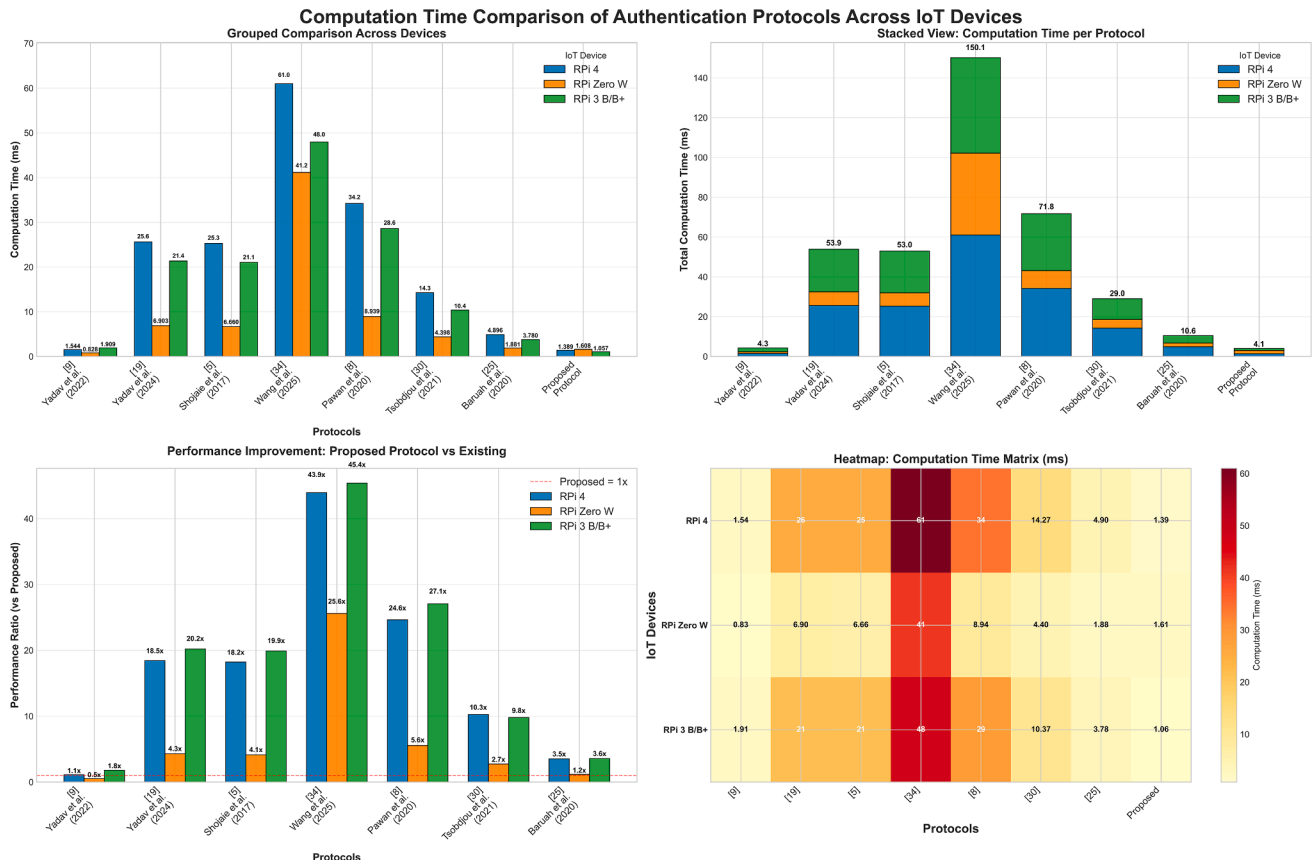
- Scalability: Evaluations used a static, single-server model. Performance under large-scale, dynamic networks with high concurrency requires further study.
- Energy Model: Energy estimates are theoretical. Empirical measurements under real-world RF conditions are necessary for validation.
- Forward Secrecy: The protocol provides forward secrecy against device compromise, but not against server key compromise combined with recorded transcripts. Stronger guarantees may require ephemeral key exchange.
- Standardization: The bespoke design must be aligned with industry standards (e.g., IEEE 802.1X, EAP) and tested for interoperability with commercial equipment.

Addressing these areas will improve the protocol's practicality and resilience for next-generation IoT deployments.

10. Conclusion

This paper presented a robust EAP-based authentication and key agreement system suitable for IoT deployments in WLAN environments. The protocol attains robust security by strategically incorporating elliptic-curve cryptography (ECC) for secure registration, lightweight symmetric-key methods for fast authentication, and a PUF-enhanced fast reconnection phase, all while minimising computational and communication costs. The design ensures mutual authentication, identity privacy, perfect forward secrecy against device compromise, and comprehensive resilience against a wide range of attacks including replay, MITM, stolen device, desynchronization, DoS, privileged insider, and traceability attacks.

Formal verification using the Real-or-Random (ROR) model and automated analysis via Scyther confirmed the protocol's robustness against known and novel attack vectors inside the Dolev-Yao adversary

**Fig. 11.** Comparison of energy consumption (mJ) by smart devices with protocols.

framework. The performance evaluations of prevalent IoT hardware platforms Raspberry Pi 4, 3 B/B+, and Zero W exhibit operational efficiency, with computation times reaching as low as 1.06 ms and communication overhead of 2560 bits, resulting in little energy expenditure (about 0.28 mJ on Raspberry Pi 4). The protocol facilitates secure registration via public channels and provides fast reconnection, rendering it both practical and scalable.

CRediT authorship contribution statement

Vijay Kumar: Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Aditya Kaushal Ranjan:** Writing – original draft, Validation, Methodology, Formal analysis, Conceptualization. **Prabhat Kumar:** Visualization, Validation, Supervision, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors are unable or have chosen not to specify which data has been used.

References

- [1] Magara T, Zhou Y. Internet of things (IoT) of Smart homes: privacy and security. *J Electr Comput Eng* 2024;2024(1):7716956.
- [2] Huang J, Kong L, Chen G, Wu M-Y, Liu X, Zeng P. Towards secure industrial IoT: blockchain system with credit-based consensus mechanism. *IEEE Trans Ind Inform* 2019;15(6):3680–9.
- [3] Ranjan AK, Kumar P. APPS: authentication-enabled privacy protection scheme for secure data transfer in internet of things. *Ad Hoc Netw* 2024;164:103631.
- [4] Cheikhrouhou O, Laurent M, Abdallah AB, Jemaa MB. An EAP-EHash authentication method adapted to resource constrained terminals. *Ann Telecommun-Ann Télécommun* 2010;65(5):271–84.
- [5] Shojai B, Saberi I, Salleh M. Enhancing EAP-TLS authentication protocol for IEEE802.11i. *Wirel Netw* 2017;23(5):1491–508.
- [6] Aboba B, Blunk L., Vollbrecht J., Carlson J., and Levkowetz H., “Extensible authentication protocol (EAP),” Technical Report, 2004.
- [7] Simon D., Aboba B., and Hurst R., “The EAP-TLS authentication protocol,” Technical Report, 2008.
- [8] Kumar P, Kumar D. A secure n-secret based client authentication protocol for 802.11 WLANs. *Telecommun Syst* 2020;75(3):259–71.
- [9] Yadav AK, Misra M, Pandey PK, Liyanage M. An EAP-based mutual authentication protocol for WLAN-connected IoT devices. *IEEE Trans Ind Inform* 2022;19(2):1343–55.
- [10] Li X, Bao F, Li S, Ma J. Flap: an efficient wlan initial access authentication protocol. *IEEE Trans Parallel Distrib Syst* 2013;25(2):488–97.
- [11] Wang L, Srinivasan B, Bhattacharjee N. Security analysis and improvements on WLANs. *J Netw* 2011;6(3):470.
- [12] Kumar A, Om H. A secure, efficient and lightweight user authentication scheme for wireless LAN. In: 2016 international conference on emerging trends in engineering, technology and science (ICETETS); 2016. p. 1–9.
- [13] Yadav AK, Misra M, Liyanage M, Varshney G. Secure and user efficient eap-based authentication protocol for IEEE 802.11 wireless lans. In: 2020 IEEE 17th international conference on mobile ad hoc and sensor systems (MASS). IEEE; 2020. p. 576–84.
- [14] Yadav AK, Misra M, Pandey PK, Kaur K, Garg S, Liyanage M. Lemap: a lightweight eap based mutual authentication protocol for IEEE 802.11 wlan. In: ICC 2022-IEEE international conference on communications. IEEE; 2022. p. 692–7.
- [15] Rana M, Shafiq A, Altaf I, Alazab M, Mahmood K, Chaudhry SA, Zikria YB. A secure and lightweight authentication scheme for next generation IoT infrastructure. *Comput Commun* 2021;165:85–96.
- [16] Chen C-M, Li X, Liu S, Wu MU-EN, Kumari S. Enhanced authentication protocol for the Internet of things environment. In: Security and communication networks. 2022; 2022. 8543894.
- [17] Alzahrani BA, Mahmood K. Provable privacy preserving authentication solution for internet of Things environment. *IEEE Access* 2021;9:82857–65.
- [18] Guo Y, Guo Y, Xiong P, Yang F, Zhang C. Deeper insight into why authentication schemes in IoT environments fail to achieve the desired security. *IEEE Trans Inf Forens Secur* 2024.
- [19] Yadav AK, Misra M, Pandey PK, Ranaweera P, Liyanage M, Kumar N. A secure authentication protocol for IoT-WLAN using EAP framework. *IEEE Trans Depend Secure Comput* 2024.
- [20] Fan C-I, Lin Y-H, Hsu R-H. Complete eap method: user efficient and forward secure authentication protocol for IEEE 802.11 wireless LANS. *IEEE Trans Parallel Distrib Syst* 2012;24(4):672–80.
- [21] Pandey A, Pant PK, Tripathi R. A system and method for authentication in wireless local area networks (WLANs). *Proc Natl Acad Sci India A: Phys Sci* 2016;86(2):149–56.
- [22] He D, Cai Y, Zhu S, Zhao Z, Chan S, Guizani M. A lightweight authentication and key exchange protocol with anonymity for IoT. *IEEE Trans Wirel Commun* 2023;22(11):7862–72.
- [23] Dey B, Vishnu S, Swarnkar OS. An efficient dynamic key based EAP authentication framework for future IEEE 802.1 x wireless LANs. In: Proceedings of the 2nd international conference on digital signal processing; 2018. p. 125–31.
- [24] El Idrissi YEH, Zahid N, Jedra M. A new EAP authentication method for IEEE802.11 wireless. *IJCSNS* 2011;11(6):1–11.
- [25] Baruah B, Dhal S. An efficient authentication scheme for secure communication between industrial IoT devices. In: Proceedings of the IEEE 11th international conference on computer and information technology; 2020. p. 1–7.
- [26] Omar HA, Abboud K, Cheng N, Malekshan KR, Gamate AT, Zhuang W. A survey on high efficiency wireless local area networks: next generation WiFi. *IEEE Commun Surv Tutor* 2016;18(4):2315–44.
- [27] Kumar V, Ranjan AK, Kumar P, et al. An EAP based Privacy Preserving Authentication and key agreement protocol for WLAN-IOT devices. PREPRINT (Version 1) 2025. <https://doi.org/10.21203/rs.3.rs-7025423/v1>.
- [28] Dolev D, Yao A. On the security of public key protocols. *IEEE Trans Inf Theory* 1983;29(2):198–208.
- [29] Canetti R, Krawczyk H. Universally composable notions of key exchange and secure channels. In: Proceedings of the international conference on theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer; 2002. p. 337–51.
- [30] Tsobdjou LD, Pierre S, Quintero A. A new mutual authentication and key agreement protocol for mobile client—server environment. *IEEE Trans Netw Serv Manag* 2021;18(2):1275–86.
- [31] Lee J, Kim G, Das AK, Park Y. Secure and efficient honey list-based authentication protocol for vehicular ad hoc networks. *IEEE Trans Netw Sci Eng* 2021;8(3):2412–25.
- [32] Braeken A. Symmetric key based 5g aka authentication protocol satisfying anonymity and unlikability. *Comput Netw* 2020;181:107424.
- [33] Grassi PA, Fenton JL, Newton EM, Perlner RA, Regenscheid AR, Burr WE, Richer JP, et al. Draft NIST special publication 800-63b digital identity guidelines, 27. National Institute of Standards and Technology (NIST); 2016.
- [34] Wang W, Xie Q, Han Z, Su C, Rodrigues JJPC, Wu K. Secure enhanced IoT-WLAN authentication protocol with efficient fast reconnection. *IEEE Trans Mob Comput* 2025.
- [35] Modarres AA, Sarbishaei G. A lightweight authentication protocol for IoT-based applications using reconfigurable noisy PUFs. *IEEE Trans on Ind Inform* 2024;20(9):11384–92.